Observer-Centric Neurocybernetics: Unifying Closed-Loop Control, Language-Game Semantics, and Hinge Hyperpriors for Brain Science

---

0. Executive Overview — Diagrams Pack

Figure 0-A. The Observer Loop (Measure → Write → Act)

Diagram (captioned flow):
World state **x_t** → **Measure M** → outcome label **ℓ_t** → **Write W** (append to Trace **T_t**) → **Act Π** (policy reads T_t) → World updates to **x_{t+1}**.

One-line mechanics (paste under the figure):
Trace update: **T_t = T_{t−1} ⊕ e_t.** (0.1)
Latching (fixedness): **E[e_t | T_t] = e_t** and **Pr(e_t | T_t) = 1.** (0.2)
Policy reads the record: **u_t = Π(T_t)**; branch diverges if the write differs. **x_{t+1} = F(x_t, u_t, T_t).** (0.3)

Rigour anchor. Latching is "conditional-expectation fixedness" of past events in the observer's filtration; policy-read causes branch-dependent futures.

Figure 0-B. Thermostat-with-a-Notebook Analogy

Diagram (captioned flow):
Read room → if cold, **write** "heat_on" in notebook → heater turns on because controller reads the notebook → tomorrow is warmer → the note can't be "unwritten" for the controller's next step.

Minimal math under the picture:
Notebook write: **e_t = "heat_on"**; **T_t = T_{t−1} ⊕ e_t.** (0.4)
Delta-certainty of your own note: **Pr(e_t = "heat_on" | T_t) = 1.** (0.5)
Why tomorrow changes: **u_t = Π(T_t)**, so **F(x_t, Π(…⊕"heat_on"), T_t) ≠ F(x_t, Π(… ⊕"off"), T′_t).** (0.6)

Figure 0-C. "Agreement You Can Trust" Panels

Layout (four tiles):

1. **CSA@3 trend** (top strip).
2. **ε heatmap** (critic-pair order sensitivity).
3. **Redundancy index** (fragments per claim in Trace).

4. **CWA lamp** (green/red “Averaging is legal?”).

One-line definitions under the panel:
Commutation on item d: **A∘B(d) = B∘A(d).** (0.7)
Order-sensitivity: **ε_AB := Pr[A∘B ≠ B∘A].** (0.8)
CSA majority (k=3 critics): **CSA@3 = mean_d[majority label unchanged by any order].** (0.9)
CWA pass rule: **CWA_OK ⇔ [CSA@3 ≥ 0.67] ∧ [p̂ ≥ α] ∧ [max ε_AB ≤ 0.05], α=0.05.** (0.10)

Operational note. These panels are the “go/no-go” for pooling across participants or sessions; if CWA fails, report **per-case (SRA)** only.

Figure 0-D. The Trace Ledger (Immutability at a Glance)

Diagram (captioned flow):
Append-only **Trace** with hash chain inside each session; daily Merkle root; dataset root for exports.

One-line formulas under the figure:
Hash chain: **h0 := 0; h_t := H(h_{t−1} ∥ canonical_json(e_t)).** (0.11)
Verify trace: **VerifyTrace(T) = 1** iff recomputed **h_T** equals stored **h_T**. (0.12)

Why it matters. Hash-chained writes make latching operational (tamper-evident past), so conditioning on your own record is well-defined for the controller Π.

Figure 0-E. System Planes (Where Each Step Runs)

Diagram (captioned boxes):
Data Plane — measurement hot-path, Trace writes, CWA-gated pooling.
Control Plane — Ô policy, tick cadence, safety gates.
Audit Plane — immutable ledger, certificate logs, exports.

Invariants under the figure:
Irreversible writes at tick **τ_k**; pooling only if **CWA.score ≥ θ**; slot conservation on buffers/ tools. (0.13)

Figure 0-F. One-Minute CWA Certificate Checklist

Diagram (checkbox list beside the CWA lamp):

□ **Three independent, non-mutating critics** (units/invariants, contradiction vs Given, trace-referencer). (0.14)
□ **Order test:** all **ε_AB ≤ 0.05** (swap critic order on a held-out batch). (0.15)
□ **Permutation p-value: p̂ ≥ 0.05** under order+phase shuffles. (0.16)
□ **CSA threshold: CSA@3 ≥ 0.67** (majority label stable). (0.17)
□ **Redundant traces:** ≥2 fragments/claim (e.g., tool log + hash). (0.18)
→ If all boxes ticked, **CWA_OK = green**; else **SRA only** (no group averages). (0.19)

Sources behind the figures (for readers without our library)

- **Latching & filtration (Figures 0-A, 0-B):** fixedness via conditional expectation; policy-read branching.
- **Commutation, CSA, CWA (Figures 0-C, 0-F):** order tests, thresholds, and the certificate recipe.
- **Ledger & planes (Figures 0-D, 0-E):** hash-chain trace, auditability, and three-plane ops.

1. Minimal Math, Human Words: The Observer Triplet

1.1 First principles (plain words → one-liners)

What is an observer?
Think "a lab team with three moves": **Measure** the world, **Write** the result into a record, then **Act** using that record. We'll call the trio $\mathcal{O}$ **= (M, W, Π)**. The running record is a **Trace** of timestamped events. This is enough to build experiments, dashboards, and controllers.

Single-line definitions (Blogger-ready).
Observer triplet: $\mathcal{O}$ **= (M, W, Π).** (1.1)
Trace as an append-only list: $T_t = [e_1,\dots,e_t] = T_{t-1} \oplus e_t$. (1.2)
Event schema (concept): $e_t = (\tau, \text{channel}, \ell_t, \text{meta})$. (1.3)

Analogy (you'll reuse it): your **lab log**. You measure, you write, and next steps are planned from what's written. You can debate *why* later, but you can't *unhappen* an entry once it's in your own log.

1.2 Latching = "conditioning on your own record"

Once you write today's outcome ℓ_t into T_t, any probability "from your point of view" is now **conditioned on T_t**. That makes the just-written event a **fixed point**: inside your frame, it's certain; and because the policy reads the trace, **tomorrow branches** based on what you wrote. This is the operational face of "collapse-as-conditioning."

One-liners.
Delta-certainty of the written label: $\Pr(\ell_t = a \mid T_t) = 1 \text{ if } a = \ell_t, \text{ else } 0$. (1.4)
Fixed-point form: $\mathbb{E}[\mathbb{1}\{\ell_t = a\} \mid T_t] = \mathbb{1}\{a = \ell_t\}$. (1.5)
Policy reads the record: $u_t = \Pi(T_t)$. (1.6)
Next step is label-selected: $x_{t+1} = F_{\ell_t}(x_t, u_t, \xi_t)$. (1.7)

Everyday picture. "Write **'exposure_ready'** in the note → the scheduler runs the exposure branch next session. If you had written **'not_ready'**, the future would route differently."

1.3 Commutation = "checks that don't interfere"

Two checks **A** and **B commute** on an item when **order doesn't matter**; then majority labels are **stable under order-swap**, and cross-observer agreement (**CSA**) rises. In practice, you test order-sensitivity ε_{AB} and require it to be small before you average anything (the **CWA** rule).

One-liners.
Commutation on x: **A ∘ B(x) = B ∘ A(x).** (1.8)
Order-sensitivity (held-out set D): **ε_AB := Pr_{d∼D}[A∘B(d) ≠ B∘A(d)].** (1.9)
CSA (3 critics, order-invariant majority): **CSA@3 = mean_d[majority label unchanged by any order].** (1.10)
CWA pass (pooling is legal): **CWA_OK ⇔ [CSA@3 ≥ 0.67] ∧ [p̂ ≥ 0.05] ∧ [max ε_{AB} ≤ 0.05].** (1.11)

Everyday picture. Three "thermometers" that don't affect each other (commute) + multiple receipts (redundant traces) → the reading is stable enough to average; if they *do* affect each other, don't average.

1.4 Implementation (copy-paste friendly)

A) 10-line templates for M, W, Π (pseudo-code).

```
# Measure
def M(x_t, T_prev):
    ℓ_t = detect_label(x_t, T_prev)        # pure readout; no edits
    return ℓ_t

# Write (append-only)
def W(T_prev, τ, channel, ℓ_t, meta):
    e_t = { "tau": τ, "channel": channel, "label": ℓ_t, "meta": meta }
    return T_prev + [e_t]                  # no overwrites; hash-chain in storage

# Policy (controller reads trace)
def Π(T_t):
    return choose_action_from_rules(T_t)   # e.g., route to branch if last two labels match
```

These mirror the "Hello-Observer" blueprint: tick increments on append; branch logic reads the written outcome; the storage layer enforces immutability with a hash chain.

B) Event JSON (minimal, extensible).

```
{
  "tau": 17,
  "channel": "task/lexical",
  "label": "exposure_ready",
  "meta": {
    "confidence": 0.82,
    "rater": "O3_trace",
    "timestamp": "2025-10-18T14:03:22Z",
    "hash_prev": "h_t-1",
    "hash_self": "h_t"
  }
}
```

Use a hash-chain (**h_t := H(h_{t−1} ‖ canonical_json(e_t))**) to make retro-edits tamper-evident; verify by recomputing the chain. (1.12)

C) 10-line ε/CSA smoke test (order-swap).

```
def eps_AB(A,B,D):
    return mean([ A(B(d)) != B(A(d)) for d in D ])

def CSA3(o1,o2,o3,D):
    from itertools import permutations
    def maj(lbls):
        return max(set(lbls), key=lbls.count)
```

```
    stable = 0
    for d in D:
        labs = set()
        for order in permutations([o1,o2,o3]):
            y = d
            for o in order: y = o(y)
            labs.add(maj([o1(d),o2(d),o3(d)]))
        stable += (len(labs)==1)
    return stable/len(D)
```

Gate pooling by **CWA_OK**; otherwise report per-case only.

1.5 Why this is rigorous (quick anchors)

- **Latching** is just conditional expectation on your own trace: once written, the event is delta-certain in-frame; tomorrow's action depends on it. Proof sketches and operator-algebra pointers are provided in the cited kit.
- **Agreement** follows when effects **commute** and writes are **redundant**; our **CSA/ε/CWA** are practical proxies with thresholds you can run today.

Neuro note (how it maps later): channels map to tasks/areas; ticks to trial epochs/cycles; Π to top-down (e.g., PFC) policy reading the trace—exactly the observer/observed hierarchy your readers expect.

Remember: *You can't unhappen what you just wrote in your lab log*. That single rule powers the math and the implementation.

2. Agreement You Can Trust: CSA & the CWA Certificate

2.1 Why this section exists (objectivity before averaging)

Before you pool participants/sessions, you need evidence that different graders/tools **agree without stepping on each other** and that **order/phase shuffles don't move your averages**. This section gives you two computable gates:

- **CSA@3** — cross-system agreement from three *independent, non-mutating* critics.
- **CWA** — a pass/fail certificate that says when simple averaging is legal.

Analogy. Three thermometers that don't interfere, plus multiple receipts in the cabinet. If swapping the reading order doesn't change the verdict and random shuffles don't budge the average, your number is "objective enough" to pool.

2.2 Single-line, Blogger-ready definitions (copy next to your figures)

CSA from order-insensitive majority (three critics, N items):
CSA@3 := (1/N) · Σ_j 1{ majority label for item j unchanged under all grader orderings }. (2.1)

Pairwise order-sensitivity (held-out set D):
ε_AB := Pr_{d~D}[A∘B(d) ≠ B∘A(d)]. (2.2)

Order/phase permutation statistic for a scalar outcome s (e.g., session score):
T_obs := | mean(s) − mean_π(s) |. (2.3)
p̂ := (1/B) · Σ_{b=1}^B 1{ | mean(s) − mean_{π_b}(s) | ≥ T_obs }. (2.4)

CWA pass rule (defaults):
CWA_OK ⇔ [CSA@3 ≥ 0.67] ∧ [max ε_AB ≤ 0.05] ∧ [p̂ ≥ 0.05]. (2.5)

Redundancy hint (receipts per claim):
Redundancy ≥ 2 fragments/claim ⇒ error shrinks by majority over fragments. (2.6)

Why this is principled. In the observer-with-trace framework, past writes are **delta-certain** in-frame; **agreement** emerges when effects **commute** and outcomes are **redundantly recorded**. CSA, ε, and CWA implement these guarantees in practice.

2.3 Implementation — minimal recipes you can run today

2.3.1 Build three pure critics (no mutations)

Start with:

- **O₁ Units/constraints** (bounds, invariants),
- **O₂ NLI/contradiction** (claim vs Given),
- **O₃ Trace auditor** (each quantitative claim has ≥2 trace fragments).
  Design rule: critics **only evaluate**, never rewrite inputs; this makes commutation testable.

2.3.2 Compute ε (order-sensitivity) and CSA (order-invariant majority)

```
def eps_AB(A,B,D):
    return sum( A(B(d)) != B(A(d)) for d in D ) / len(D)

from itertools import permutations

def CSA3(O1,O2,O3,D):
    stable = 0
    for d in D:
        maj = lambda ys: max(set(ys), key=ys.count)
        base = maj([O1(d),O2(d),O3(d)])
        ok = True
        for order in permutations([O1,O2,O3]):
            y = [o(d) for o in order]
            if maj(y) != base: ok = False; break
        stable += ok
    return stable/len(D)
```

Targets: **max ε_AB ≤ 0.05**, **CSA@3 ≥ 0.67** (2-of-3 stable under all orders).

2.3.3 Permutation test (order + phase)

- **Order**: randomly permute critic order per item.

- **Phase**: circular-shift within-session segments to ensure timing/phase does not bias the mean.
  Compute **T_obs** and **p̂** as in (2.3)–(2.4). Use **B = 1000** permutations as a default.

2.3.4 Decide pooling

If **CWA_OK** by (2.5), you can average across cases; otherwise **report per-case (SRA)** and fix the pipeline (separate critic inputs, add trace redundancy, or split monolithic critics).

2.4 Dashboard spec (one screen your team will actually use)

Top strip — CSA@3 trend. 7-run EMA; target ≥ 0.67.
Middle left — ε heatmap. Cells alert > 0.05; quarantine a noisy critic if a cell spikes.
Middle right — Redundancy index. Mean fragments/claim (goal ≥ 2.0).
Bottom — CWA lamp. Green if (2.5) holds; red → "SRA only; add redundancy; refactor critics."

Ops notes: log CSA, ε-matrix, redundancy; when **CSA EMA < 0.67**, cool interventions and re-verify; treat **δ-max > 0.05** as an incident (order interference).

2.5 Troubleshooting & good habits

- **ε hot cell (> 0.05):** critics share inputs or one mutates text. **Fix:** make critics pure; ensure NLI reads only the [Given]/Trace, not post-repair drafts; or split one big critic into two narrow ones.
- **CSA low (< 0.67):** tighten definitions; improve critic separation; add trace redundancy to reduce ambiguity.
- **Permutation p̂ low:** order/phase bias exists; stabilize timing (phase) or decouple graders; do not average.
- **Remember the principle:** objectivity = **commuting effects + redundant records**; that's the operational mirror of the formal agreement theorems.

Where the math lives (optional reading): proof sketches for latching and agreement (delta-certainty; AB-fixedness) and the invariance view (Collapse-Lorentz) that keeps agreement meaningful under frame remaps.

Deliverables you now possess: ① precise CSA/ε/CWA definitions (2.1–2.5), ② a 20-line implementation you can paste into your analysis notebook, ③ a dashboard layout with thresholds and ops rules. This is enough to gate pooling in EEG/MEG/fMRI/ECoG/TMS pipelines on day one.

3. Mapping to Brain Experiments (EEG/MEG/fMRI/ECoG/TMS)

3.1 One-page crosswalk (what maps to what)

Channels θ = task contexts / receptive fields; sensor-specific features are simple readouts of a latent neural state.
Single-line: y_modality(t) = Φ_modality[x(t)] with **modality ∈ {EEG, MEG, fMRI, ECoG}.** (3.1)

Ticks τ = trial epochs / cycles (e.g., cue→stimulus→response); writes happen at epoch boundaries.
Frame maps = area-to-area remaps (retinotopic→object-centric; parcel A→parcel B) used to compare observers across regions. **Meta-observer** logic lives in PFC (policy/frames); **binder** role is plausibly claustral.

PFC as meta-observer (top-down policy gain).
g_neuro := strength(PFC → target) via DCM/Granger or TMS-evoked transfer. (3.2)

Claustrum as binder (integration hub).
Binding cascade index: **CI := coh(V1↔IT) · coh(IT↔PFC)** (lag-ordered). (3.3)
Claustral synchrony: **CL_sync := mean_coh(claustrum ↔ cortex)** during integrated states − baseline. (3.4)

Stability dial (neural mirror).
Δ_neuro := g_neuro · β_neuro − γ_neuro. (3.5)
β_neuro := network switch rate (EEG microstates/HMM); **γ_neuro := 1 ÷ τ_recover(neural).** (3.6)

Objectivity gates for pooling.
Cross-modality agreement: **CSA_neuro := fraction of items where {EEG, MEG, fMRI critics} agree under order-swap.** (3.7)
Pooling is legal iff **CWA_neuro = [CSA_neuro high] ∧ [permutation p̂ ≥ α].** (3.8)

Why these mappings: they are the neural mirrors of the observer-with-trace calculus (latching, commuting effects, redundancy) and align with independent PFC/claustrum predictions in the attached brain-science paper.

3.2 Protocol blocks you can run this week

(A) Order-swap rater pipeline (build ε and CSA first)

Critics (pure, no mutation):
• **EEG critic:** microstate/HMM labeler on sensor or parcel series.
• **MEG critic:** phase-locking / coherence-cascade scorer.
• **fMRI critic:** ROI pattern-match / directed-gain readout (for g_neuro).
Compute **ε_AB := Pr[A∘B ≠ B∘A]**, target **max ε_AB ≤ 0.05**; then **CSA@3 ≥ 0.67**. (3.9)

Permutation test (order + phase): circular-shift session segments; shuffle critic order; require **p̂ ≥ 0.05** before averaging across participants. (3.10)

Dashboard: top CSA@3 trend; middle ε heatmap; bottom **CWA** lamp; quarantine a noisy critic if a cell > 0.05. (3.11)

Analogy. Three thermometers that don't jostle each other, plus a shuffle test to prove the average won't move when you re-order the readings.

(B) Redundant trace fragments (text + time-locked sensor)

Every reported neural claim gets ≥ 2 fragments: (i) **time-locked sensor feature** (e.g., bandpower window ID), (ii) **textual event** (trial note / button log).
Hash-chain for tamper-evidence: h₀ := 0; h_t := H(h_{t−1} ∥ canonical_json(e_t)). (3.12)
Pooling only with **CWA_neuro = green**; else per-case (SRA) reporting. (3.13)

Analogy. Keep both the pharmacy receipt and the credit-card slip; the hash makes back-dating obvious.

(C) Pre-registered endpoints (clear dials + falsifiers)

1. **Δ-dial perturbation (causal).**
   Design: single-pulse **dlPFC TMS** to modulate **g_neuro**; paced breathing to raise **γ_neuro**; free-association task to raise **β_neuro**.
   Endpoint: **Δ_neuro** falls when **g_neuro**↓ or **γ_neuro**↑; hitting-time (CUSUM) shifts right. Fail = no change → revise mapping. (3.14)

2. **Binding cascade with claustral check.**
   Design: narrative-integration task with **MEG** (CI) + **fMRI/SEEG** (CL_sync); optional **tFUS/SEEG claustral** perturbation where ethical/available.
   Endpoint: CI↑ during integration; brief claustral disruption drops CI and **CSA_neuro** across modalities. Fail = no CI change → revise hub model. (3.15)

3. **Transference direction field (A_θ) bias.**
   Design: evoke **past-template** vs **present-cue**; compute **κ̂_neuro := bias(PFC→template) − bias(PFC→present)**.
   Endpoint: labeling/time-indexing reduces **κ̂_neuro** and **Δ_neuro**. Fail = κ̂_neuro stays high at low TI with high CSA → bridge incomplete. (3.16)

Pedagogy rule: "one dial, one number, one test." (g, β, γ, Δ, CI, CL_sync, CSA, CWA). (3.17)

3.3 Modality cheat-sheet (how to compute each dial)

EEG (β_neuro, γ_neuro, CSA role).
• **β_neuro:** fit K-state microstates/HMM; **β_neuro = transitions/min.** (3.18)
• **γ_neuro:** present probe (aversive image or single-pulse TMS); **γ_neuro = 1 ÷ τ_recover** in alpha/DMN signal. (3.19)
• **Critic:** microstate labeler for CSA trio.

MEG (CI, CSA role).
• **CI:** compute **coh(V1↔IT)** and **coh(IT↔PFC)** with lag (sensory→assoc→PFC); multiply. (3.20)
• **Critic:** phase-locking / coherence-cascade scorer.

fMRI (g_neuro, CL_sync, CSA role).
• **g_neuro:** DCM/Granger, or TMS-evoked ROI transfer from PFC. (3.21)
• **CL_sync:** average claustrum↔cortex synchrony in integrated vs baseline windows. (3.22)
• **Critic:** ROI pattern-match / directed-gain readout.

ECoG (all of the above where available).
• High-SNR confirmation of CI and CL_sync; optional intra-operative claustral mapping in rare cases. (3.23)

TMS/tFUS (perturbation handles).
• **dlPFC TMS** for **g_neuro**; **tFUS SEEG-guided** claustral ping for CL_sync (where ethical).

Report with CSA/CWA gates. (3.24)

Independent alignment (reader-friendly): PFC as meta-observer and claustrum as binder are also argued independently in the attached theory paper, yielding testable predictions (TMS to PFC should alter reported experience; claustral changes should affect binding). (3.25)

3.4 Safety, governance, and reporting (the boring bits that save you)

CWA discipline. Never average unless **CWA_neuro** is green; otherwise publish per-case (SRA). Include **CSA@3**, **max ε_AB**, **p̂**, and fragments/claim in every report footer. (3.26)

Auditability. Export the dataset root hash + session hash tail; keep consent proof and role-based access logs. (3.27)

Runbook thresholds. Accept **CSA@3 ≥ 0.67**, **max ε_AB ≤ 0.05**; if CSA EMA dips, cool **g_neuro** interventions and re-verify before resuming pooling. (3.28)

3.5 Plain-speech recap (why this is practical)

- You **treat PFC as the policy knob** (g_neuro), **claustrum as the binder** (CL_sync), and **CI as the staircase** that climbs from sensation to integration.
- You **prove objectivity** with **CSA/ε/p̂** before you average anything.
- You **write everything to an immutable trace** so "you can't unhappen what you just wrote," and your next-step policy depends on that write.
- Each dial is **one line to compute** and **one test to pass**, with falsifiers pre-registered—no PhD math required.

Where deeper theory lives (optional): the observer-with-trace invariants (latching; AB-fixedness under commuting effects; certificate-gated pooling) and their operationalization in ObserverOps/ESI.

Analogy. It's a kitchen: PFC is your head chef (policy), claustrum is the expo station (binder), CSA is your blind taste-test agreement, and CWA is the rule "don't publish the menu average until three tasters agree and the order of tasting doesn't change the verdict."

4. Closed-Loop Control for Neuroscience

4.1 Treat Π as a controller (the loop we actually run)

Closed loop = read signals, write to trace, choose an action, world updates.
State update: **x_{t+1} = F(x_t, u_t, T_t) + η_t.** (4.1)
Readout: **y_t = H(x_t) + ε_t.** (4.2)
Controller (policy): **u_t = Π(T_t, y_t, c_t)** with context **c_t** (task, safety, consent). (4.3)

Why trace matters. Writes make the just-observed event delta-certain in-frame, so Π *must* condition on **T_t**; that is latching as conditional expectation in the formal observer model.

4.2 The stability dial (push × echo − buffer)

We run control with one explainable dial; compute it from behavior/physio and show it live.
Stability discriminant: $\Delta := \hat{g} \cdot \hat{\beta} - \hat{\gamma}$. (4.4)
Guidance slope: $\hat{g} := \mathrm{cov}(r_t, s_t) \div \mathrm{var}(r_t)$ (reframe strength → stance shift). (4.5)
Amplification rate: $\hat{\beta} := (1 \div T) \cdot \Sigma_t [\, \mathrm{jumps}_t \div \mathrm{minutes}_t \,]$. (4.6)
Damping buffer: $\hat{\gamma} := 1 \div \tau_{\mathrm{recover}}$ (time to baseline after a spike/perturbation). (4.7)

Neuro mirrors. $\hat{g}$ ↔ PFC→target gain (DCM/Granger or TMS-evoked transfer); $\hat{\beta}$ ↔ microstate/assembly switch rate; $\hat{\gamma}$ ↔ recovery constant of bandpower/DMN metrics.

4.3 Safe actuation: only when evidence says "OK"

CWA certificate gate (pooling/actuation legality):
$\mathrm{CWA_OK} \Leftrightarrow [\, \mathrm{CSA@3} \geq 0.67 \,] \wedge [\, \max \varepsilon_{AB} \leq 0.05 \,] \wedge [\, \hat{p} \geq 0.05 \,]$. (4.8)
Action arming: $u_t := u_t$ if CWA_OK else u_{hold}. (4.9)

Redundant receipts. Require ≥ 2 independent fragments per claim (e.g., sensor window + text tag) in T_t; hash-chain all writes.
Hash chain: $h_0 := 0;\ h_t := H(h_{t-1} \,\|\, \mathrm{canonical_json}(e_t))$. (4.10)

4.4 Controller baselines you can deploy this week

(A) PID (robust, model-light)

Error to a target dial Δ^*: $e_t = \Delta^* - \Delta_t$. (4.11)
PID law (discrete): $z_t = z_{t-1} + e_t \cdot \Delta t;\ u^{\mathrm{raw}}_t = K_p\, e_t + K_i\, z_t + K_d\, (e_t - e_{t-1}) \div \Delta t$. (4.12)
Safeguards: $u_t = \mathrm{sat}_U(\, \mathrm{rate_limit}(\, u^{\mathrm{raw}}_t \,)\,)$. (4.13)

Defaults. Start tiny: K_p small, K_i off (0) until CSA is stable, K_d for overshoot control; anti-windup: freeze z_t while u_t saturates. Gate execution by (4.9).

(B) LQR (when you can identify a local model)

Local ID (rolling window): $x_{t+1} \approx A\, x_t + B\, u_t$ by ridge regression on recent pairs. (4.14)
LQR objective: $\min \Sigma_t [\, x_t^{\mathsf{T}}\, Q\, x_t + u_t^{\mathsf{T}}\, R\, u_t \,]$. (4.15)
Policy: $u_t = -K\, x_t$, with $K = \mathrm{dlqr}(A,B,Q,R)$ (discrete Riccati). (4.16)

Practice tip. Keep Q on Δ-related coordinates, make R large for invasive actuators (TMS/tFUS), and still arm by CWA_OK.

4.5 Safe action sets, rate limits, and consent

Hard bounds (per actuator): $U_{\mathrm{safe}} = \{\, u : |u| \leq u_{\max} \wedge |u - u_{t-1}| \leq \rho_{\max} \wedge \mathrm{IPI} \geq \tau_{\mathrm{ref}} \,\}$. (4.17)
Consent gate: $\mathrm{Allow}(u, \mathrm{action}, \mathrm{resource}) = 1$ iff role/consent policy passes; else $u = 0$. (4.18)

Guardrails to log: CWA light, CSA@3 EMA, ε-matrix, redundancy index, consent state, Δ trend; auto-cool g-moves when CSA EMA dips.

4.6 Implementation sketch (neurofeedback, TMS/tDCS/tFUS, task gating)

```
loop each epoch:
  y_t ← H(x_t)  # EEG/MEG/fMRI/ECoG features
  T_t ← append(e_t) with hash-chain                # redundant receipts
  CSA, ε, p̂ ← update_agreement_metrics(D_holdout) # cheap batch
  CWA_OK ← (CSA≥0.67 and max ε≤0.05 and p̂≥0.05)

  ĝ, β̂, γ̂← estimate_slopes_and_times(y_t, T_t)  # (4.5)-(4.7)
  Δ_t ← ĝ*β̂ - γ̂                                  # (4.4)

  u_cmd ← controller(Δ_t, Δ*)                      # PID or LQR
  u_t ← (CWA_OK ? clamp_to_U_safe(u_cmd) : u_hold) # (4.9), (4.17)

  actuate(u_t)  # neurofeedback gain, TMS intensity/timing, task route
  log(Δ_t, CSA, ε, p̂, u_t, hashes)
```

This is the ObserverOps + ESI sidecar pattern: a tiny control layer, CSA-gated, with reproducible hashes and incident playbooks.

4.7 Audio channel-strip analogy (for the whole team)

Steering (Π focus) sets what you try to move (Δ target). **Drive (ĝ)** pushes; if the room is **echo-y (β̂)** you'll ring; the **fence (γ̂, U_safe, consent)** keeps you out of trouble; **routing (CWA_OK)** only lets signal pass when meters say the chain is behaving.

4.8 When to stop or switch strategies (failure modes → actions)

- **ε spike (> 0.05)** ⇒ critics interfering; split inputs, refactor to pure effects, retest; hold actuation.
- **CSA EMA < 0.67** ⇒ cool guidance (**ĝ**↓), add redundancy, re-arm later.
- **p̂ < 0.05** under order/phase permutations ⇒ phase/order bias; stabilize timing or per-case SRA only.

Bottom line. A simple Δ-dial with PID/LQR, armed by a CWA certificate and fenced by consent/limits, turns "observer-with-trace" into a safe, reproducible control loop for neurofeedback and perturbation studies—no heavy math required, but with rigorous anchors when you need them.

5. Language-Game Semantics in the Lab (Wittgenstein, Operationalized)

5.1 What we measure (plain words → one-liners)

Picture → world fit. A statement "fits" when its internal structure can be mapped into a scene-graph of the world without breaking relations; with noise, we score approximate fits. **Truth-as-correspondence (feasibility): ∃ map h: V → D such that every pictured tuple $(v_1,\ldots,v_m)$ in Q_j lands in a world tuple $(h(v_1),\ldots,h(v_m))$ in R_j.** (5.1)

Empirical fit (with partial observability): Fit(h) := (1/|Obs|) · Σ_{o∈Obs} 1{ h respects o's constraints }. (5.2)

Meaning = use (as policy). "What a word means" is the **policy component** that makes cooperative interaction go well; we estimate it from logs (inverse RL) and keep it stable with trust-region steps.
Meaning-as-policy: π*(a | x,u) := argmax_π E[Σ_t γ^t · u(x_t,a_t) | π]. (5.3)
IRL objective (max-ent form): θ̂ := argmax_θ [E_{data}[φ] − log Z(θ)], π_θ(a|x) ∝ exp(θ·φ(x,a)). (5.4)
Trust-region update: KL(π_{new} ‖ π_{old}) ≤ δ, θ_{new} = θ_{old} + η·ĝ with KL-penalty. (5.5)

Hinge certainty (hyperpriors + switching costs). Background certainties ("hinges") change **only** when evidence is overwhelming.
BF stream & stop rule: Λ_t := Σ_{k≤t} log BF_k; switch hinge iff Λ_t ≥ λ (cost-of-change). (5.6)

Why this is rigorous here. These restatements (CSP homomorphism; equilibrium/IRL; Bayes-factor stopping) are the core program of *Wittgenstein, Operationalized*, designed for estimation and falsification.

Analogy. Your statement is a **subway map**; the world is the **city**. A good map "fits" when station-to-station relations line up with streets and lines. "Meaning = use" is how commuters actually ride. Hinges are those **non-negotiable landmarks** you only revise when dozens of photos and GPS tracks prove the map is wrong.

5.2 Picture→world fits you can compute (CSP/ILP templates)

Objects. World $\mathcal{W}$ **= (D; R₁,…,R_k)**, Picture $\mathcal{P}$ **= (V; Q₁,…,Q_k)**, candidate map **h: V→D**.
Homomorphism constraint: (v₁,…,v_m)∈Q_j ⇒ (h(v₁),…,h(v_m))∈R_j. (5.7)

Encoding (CSP). Variables = picture nodes; domains = candidate entities; constraints = relation arcs; solve by AC-3/AC-4; escalate to ILP/MaxSAT with slacks when noisy.
Noisy ILP (sketch): maximize Σ w_e z_e subject to: relation-preservation with slack s_e; z_e∈{0,1}; s_e ≥ 0; budget Σ s_e ≤ ϵ. (5.8)
Complexity note. Worst case NP; practical with bounded treewidth & small arities; keep pictures sparse.

Outputs to log: Fit@top-1, **#violations**, **nDCG over candidate h**, and **ECE** (calibration) for Fit confidence.

5.3 Meaning-as-use via IRL (trust-region loop you can run)

Data. Interaction log **D = {(x_t, u_t, a_t, r_t)}** with context **u_t** (task, norms), actions **a_t**, outcomes **r_t**.
Estimator. Max-ent IRL for θ over features φ; policy **π_θ(a|x) ∝ exp(θ·φ(x,a))**. (5.9)
Stability. Use a **KL trust region**: **maximize J(θ) − β·KL(π_θ ‖ π_{θ_old})**, giving monotone surrogate decrease. (5.10)

One-screen loop (pseudocode).

```
repeat:
  θ ← θ + η * grad_IRL(D, θ)           # doubly robust off-policy eval
  enforce KL(πθ || πθ_old) ≤ δ         # trust region (clip or penalty)
  calibrate πθ with temperature τ*      # see 5.4 (ECE)
  log AUC@use, ΔU vs. baseline, κ/α, ECE, BF(hinges)
until ΔU, ΔExclusivity plateau
```

What you publish. ΔU (utility lift), **out-of-context robustness**, **κ/α** (inter-rater agreement), and **calibration** (ECE↓). Implementation/runtime notes are spelled out in the paper's Appendix B.5.

5.4 Calibration & evaluation harness (ECE, κ/α, BF)

ECE (expected calibration error, b bins).
ECE := Σ_{b=1}^B (|S_b|/N) · | acc(S_b) − conf(S_b) |. (5.11)

Cohen's κ (two raters).
κ := (p_o − p_e) ÷ (1 − p_e), p_o = observed agreement, p_e = chance agreement. (5.12)

Krippendorff's α (K raters, any scale).
α := 1 − (D_o ÷ D_e), D_o = observed disagreement, D_e = expected by chance. (5.13)

Bayes-factor trajectory for hinges.
Λ_t := Σ_{k≤t} log(p(data_k | H₁) ÷ p(data_k | H₀)); switch if Λ_t ≥ λ. (5.14)

CWA gate before reporting pooled scores.
CWA_OK ⇔ [CSA@3 ≥ 0.67] ∧ [max ε_AB ≤ 0.05] ∧ [p̂ ≥ 0.05]. (5.15)

5.5 End-to-end implementation (drop-in blocks)

(A) Picture fit (CSP→ILP).

1. Build **scene-graph** (entities/relations). 2) Encode variables/domains/constraints. 3) AC-3 prune; if infeasible, run ILP/MaxSAT with slack budget ϵ. 4) Return **Fit**, **ĥ**, **violations**.

(B) Meaning (IRL).

1. Extract φ from usage (turn-level features). 2) Train **π_θ** with doubly-robust off-policy eval. 3) Apply **trust-region** (δ ≈ 0.01–0.1). 4) Calibrate with temperature/Platt, report **ECE**.

(C) Hinges (BF streams).

1. Declare hinge variables (what cannot drift without evidence). 2) Maintain **Λ_t** per (5.14). 3) Revise only if **Λ_t ≥ λ** (e.g., λ ≈ 3–5 nats). 4) Log switch decisions with evidence windows.

(D) Agreement & safety.
Run **CSA@3**, **ε_AB**, **permutation p̂** each batch; only publish pooled metrics or actuate policies when **CWA_OK**. (5.15)

(E) Repro & governance.

Release tool cards, seeds, and exact CSP/IRL configs; attach data-minimization, privacy, and misuse mitigations (rate limits, audit logs, kill switch).

5.6 Subway-map analogy (why teams "get it" fast)

- **Picture fit** = lining up the **map** with the **city** (station links ≙ relation constraints).
- **Meaning = use** = how riders actually **choose routes** (policy learned from commuting logs).
- **Hinges** = **landmarks** you only move after overwhelming photo/GPS evidence (BF threshold).
- **CWA certificate** = "three surveyors agree; order of checks doesn't change the verdict; shuffles don't move the average" → *you may publish the citywide average*.

Where to look if readers want proofs or deeper scaffolding

- **Picture/Use/Hinges formalism + code notes:** *Wittgenstein, Operationalized* §§2–6, §9, App. B.5 (CSP/ILP encoding, IRL trust-region runtime, metrics pack).
- **CSA/ε/CWA gate & dashboards:** *ESI* §8; *Freud→Control* §8 + App. E (production thresholds, one-figure board).
- **Observer foundations (why latching/commutation matter):** *Self-Referential Observers* Ch. 6 (conditional-expectation fixedness; commuting effects).

Bottom line. With these blocks, a lab can **fit pictures**, **learn use-policies**, **guard hinges**, and **publish calibrated, agreement-vetted results**—all with single-line equations and drop-in templates.

6. Psychoanalytic Knobs as Safe Operators (Clinically Intuitive, Mathematically Clean)

6.1 Operator dictionary (one-liners + what they do)

Repression = tighten the fence ($\hat{S}$_tight).
$\hat{S}$_tight[x] := $\hat{S}$[x] + Λ · χ_taboo(x), with Λ ≥ 0 (mask χ_taboo penalizes forbidden regions). (6.1)
Δ effect. Damping $\gamma\hat{}\uparrow$ (less spill), but hidden pressure may re-route unless you also redirect **V** (risk of delayed $\beta\hat{}$ rebound). Micro-vignette and mixing-board analogy appear below.

Isolation = channel decoupling (U_⊥).
C_iso := U_⊥(C) = diag(C) (drop cross-talk; keep only self-couplings). **C** is a coupling matrix among narratives/symptoms. (6.2)
Δ effect. Cuts amplification $\beta\hat{}\downarrow$ (fewer runaways); overused, integration stalls (**Δ** → **0**).

Projection = frame rotation (R_θ).
x′ = R_θ · x (choose θ to re-aim "self ↔ other" axes for the active affect). (6.3)
Δ effect. Can mis-aim guidance $\hat{\mathbf{g}}$; rotate slowly with $\gamma\hat{}\uparrow$ pacing to prevent overshoot.

Sublimation = path redirection (edit V into a safe basin).
V_id^→(x) := V_id(x) + μ · B_safe(x), μ>0; force becomes −∇V_id^→ = −∇V_id −

μ∇B_safe. (6.4)
Δ effect. Pulls energy into a safe trench (effective **ĝ↓** on risky routes, **γ̂↑** via structured outlets).

> **Analogy (mixing board).** Gate = repression; Mute/bus split = isolation; Pan = projection; Send-to-reverb/aux = sublimation. (Card repeated in §6.6.)

6.2 How they combine in one closed-loop step (the "plug-in chain")

Edited update with defenses active:
x_{t+1} = F(x_t, Ω̂_self(x_t), 𝕋_t) − ∇V_id^→(x_t) − Ŝ_tight[x_t], with couplings replaced by C_iso = U_⊥(C). (6.5)
Read left→right like a channel strip: steering (**Ω̂_self**) → drive pull (**−∇V**) → fence (**Ŝ**) → routing (**C_iso**).

Why this is principled. The *observer-with-trace* model makes the write **e_t** delta-certain in-frame (latching), so changing **Ω̂_self** or the operator schedule genuinely changes the next-step map **F** (collapse-as-conditioning). Agreement and gating rely on commuting effects and redundant records.

6.3 Scheduling & safety (what you actually run)

Operator schedule (piecewise, tick-based):
σ(t) := { λ_S(t), λ_⊥(t), θ(t), μ(t) } controls **Ŝ_tight, U_⊥, R_θ, V_id^→**. (6.6)
Arming rule: Apply σ(t) only if **CWA_OK ⇔ [CSA@3 ≥ 0.67] ∧ [max ε_AB ≤ 0.05] ∧ [p̂ ≥ 0.05].** (6.7)
Safe set: U_safe = {σ: |Δσ| ≤ ρ_max ∧ bounds(g,β,γ) respected}. (6.8)

Runtime heuristics (drop-in):
• Start with **μ (sublimation)** and **small Λ (Ŝ_tight)**; schedule **U_⊥** briefly; apply **R_θ** as micro-rotations.
• Never hard-gate (Λ big) if your CSA is low or ε hot; stabilize critics first, then escalate.

6.4 30-second micro-cases (before/after)

Case A — "Topic clips, then leaks elsewhere" (Repression).
Before: "When dad visits I… (breaks off)." After **Ŝ_tight↑**: "It's nothing." (Topic suppressed; later shows as insomnia.) *Δ:* **γ̂↑**, risk **β̂** rebound—pair with **V_id^→** (exercise/art trench). (6.9)

Case B — "Thought→body spiral" (Isolation).
Before: "Thinking about mom tightens my chest; I cancel plans." After **U_⊥**: "I can think about mom without the chest stuff." *Δ:* **β̂↓**; schedule re-coupling once **γ̂** improves. (6.10)

Case C — "Blame flips outward" (Projection).
Before: "I'm furious at myself." After **R_θ**: "My team keeps sabotaging me." Then slowly rotate back toward the true source while pacing (**γ̂↑**). (6.11)

Case D — "Doom-scroll → morning sprint" (Sublimation).
Before: "When tense I doom-scroll." After **V_id^→**: "When tense I walk and voice-note my short story." *Δ:* **ĝ↓** on risky path, **γ̂↑** via structure. (6.12)

All together (one-week edit plan).
$\hat{S}$_tight↑ on phone use after 23:00; **U_⊥** between "critique→doom-scroll" for 2 weeks; **R_θ** to "anger at process, not self/others"; **V_id^→** adds 07:30 writing trench. **Observed: Δ = $\hat{g}\cdot\hat{\beta}$ − $\hat{\gamma}$** falls (risk→safe). (6.13)

6.5 When not to use hard gates (and what to do instead)

Avoid hard $\hat{S}$_tight or big R_θ when evidence is brittle. Use the **CWA certificate** first; if red, report per-case (SRA) and fix pipeline: make critics pure, reduce order-sensitivity **ε**, add redundant trace fragments. (6.14)
If ε hot (>0.05): critics are interacting—split inputs, re-run ε; only then re-arm σ(t). (6.15)
If CSA low (<0.67): defer averaging/hard gates; raise redundancy and tighten definitions; re-test. (6.16)

6.6 Pedagogy cards (paste alongside your plots)

One-liners to remember.
Repression: **$\hat{S}$_tight[x] := $\hat{S}$[x] + Λ·χ_taboo(x).** (6.1)
Isolation: **C_iso := U_⊥(C) = diag(C).** (6.2)
Projection: **x′ = R_θ·x** (re-aim affect vector). (6.3)
Sublimation: **V_id^→ = V_id + μ·B_safe; −∇V_id^→ = −∇V_id − μ∇B_safe.** (6.4)
Closed-loop with defenses: **x_{t+1} = F(…) − ∇V_id^→ − $\hat{S}$_tight, with C→C_iso.** (6.5)

Analogy card. Gate = repression; Mute/bus split = isolation; Pan = projection; Send-to-reverb/aux = sublimation.

6.7 Why this is rigorous (anchor in one breath)

- **Latching (delta-certainty).** Once written to the trace, an event is a fixed point of conditional expectation; policies that read the trace make futures branch by what you just wrote. (Operator-algebra wording and proof sketch supplied.) (6.17)
- **Agreement (AB-fixedness).** Commuting checks + redundant records produce order-invariant majorities (high CSA) and legal pooling (CWA). Your ε/CSA/CWA gates are the computable proxies. (6.18)
- **Production guardrails.** Defaults, dashboards, and incident playbooks for CSA/ε/trace redundancy come from the deployment blueprints (ESI + clinical workflow). (6.19)

Bottom line. Treat defenses as **scheduleable operators** on **V, $\hat{S}$, $\hat{\Omega}$_self, C**; compute **Δ = $\hat{g}\cdot\hat{\beta}$ − $\hat{\gamma}$**; arm actions only when **CWA_OK**; and keep every decision tied to redundant, hashed trace fragments. That's a clinically intuitive *and* mathematically clean way to use "psychoanalytic knobs" safely in a modern, closed-loop lab.

7. Data & Trace Engineering

7.1 Non-negotiables (design goals → one-liners)

Append-only trace. You never overwrite; you only append verifiable events.

$T_t = T_{t-1} \oplus e_t$. (7.1)

Event shape (minimum viable).
$e_t = (\tau_t, \text{label}_t, \text{meta}_t)$. (7.2)

Time-lock within a session.
$\tau_t \geq \tau_{t-1}$ and **iso_time_t strictly increasing.** (7.3)

Hash-chain integrity (tamper-evident).
$h_0 := 0$; $h_t := H(h_{t-1} \| \text{canonical_json}(e_t))$. (7.4)

Merkle roll-ups for fast audit.
$R_{day} := \text{MerkleRoot}(\{ h_t : e_t \text{ on that day} \})$; $R_{dataset} := \text{MerkleRoot}(\{ R_{day} \})$. (7.5)

Verifiability.
VerifyTrace(T) = 1 iff recomputed **h_T** equals stored **h_T**. (7.6)

Redundancy (receipts per claim).
$\text{fragments_per_claim} \geq 2$ (e.g., sensor window ID + text note). (7.7)

RBAC + consent gate.
Allow(u, action, resource) = 1 iff **[role(u) $\in$ policy[action,resource] $\wedge$ consent(resource) $\geq$ level(action)].** (7.8)

Publish/act only when evidence is stable.
PublishOK $\Leftrightarrow$ CWA_OK $\wedge$ VerifyTrace(T) $\wedge$ consent $\geq$ metrics_only. (7.9)

Idempotent writes (retries are no-ops).
Append(T,e) does nothing if **event_id(e)** already exists with the same **hash**. (7.10)

Analogy. Lab **EHR** meets **Git**: every entry is a commit; daily and dataset Merkle roots are your release tags.

7.2 JSON schemas (paste-and-run)

(A) `trace.jsonl` — append-only ledger (one line = one event).

```
{
  "session_id": "S001",
  "event_id": "S001_e07",
  "iso_time": "2025-10-18T15:02:11Z",
  "label": "exposure_completed_tiny",
  "meta": { "SUDS_drop": 3, "rater_id": "r_lin", "device_id": "eeg_04" },
  "prev_hash": "0000000000",
  "hash": "8f7a...3b"        // H(prev_hash ‖ canonical_json(e_t))
}
```

Fields and hash recipe match the governance appendix; do not overwrite—append a `correction_of` when needed.

(B) `segments.jsonl` — time-locked features (EEG/MEG/etc.).

```
{
  "session_id": "S001",
  "segment_id": "S001_seg03",
  "start_sec": 720.0,
  "end_sec": 900.0,
```

```
  "rater_id": "r_kim",
  "device_id": "meg_02",
  "features": { "r_strength": 2, "stance_shift": 1, "assoc_jumps": 6, "minutes": 3.0 }
}
```

Keep rater/device IDs explicit for audit and bias checks.

(C) `graders_0*.jsonl` — pure critics (for CSA/ε/CWA).

```
{
  "item_id": "S001_seg03::avoidance_high",
  "order": ["O1","O2","O3"],
  "vote": 1,
  "confidence": 0.82,
  "rater_id": "o1_unitizer",
  "notes": "Units consistent; boundaries OK."
}
```

Critics are **pure evaluators** with **disjoint inputs** so order swaps don't change votes (ε small).

7.3 ID & hashing conventions (cut-sheet)

Event IDs. event_id := session_id ++ "_e" ++ k, **k** = 2-digit counter. (7.11)
Device IDs. device_id := modality ++ "_" ++ index (e.g., `eeg_04`, `tms_01`). (7.12)
Rater IDs. rater_id ∈ {human initials, critic name}; store mapping in Ops vault. (7.13)
Hashing. hash := H(prev_hash ‖ canonical_json(e_t)); canonical JSON means stable key order + UTF-8; record **prev_hash** on every line. (7.14)
Daily/Master roots. Write **R_day** and **R_dataset** to `meta.json` after batch jobs. (7.15)

7.4 Minimal APIs (append, query, certify)

Append (idempotent).
Append(T,e) → return **h_new**; if **event_id(e)** exists with same **hash**, return existing **h** (no-op). (7.16)

Query (by time/range/label).
Query(T; t0,t1, labels, device_id, rater_id) → stream of events; includes `prev_hash`/`hash`. (7.17)

Certify (export gate).
Certify(range) returns `{ VerifyTrace=0|1, R_day[], R_dataset, CWA_OK }`. (7.18)

RBAC wrapper.
Serve(action, resource) only if **Allow(u, action, resource)=1**; otherwise deny + audit log. (7.19)

One-line publish rule.
Export allowed iff **PublishOK** in (7.9); include **dataset_root_hash** in the report footer. (7.20)

7.5 Ops runbook (daily/weekly you can actually do)

Per session (≈ 20 min). Ingest → segment → compute $\hat{\mathbf{g}}$, $\hat{\boldsymbol{\beta}}$, $\hat{\boldsymbol{\gamma}}$, $\bar{\boldsymbol{\Delta}}$ → run **CSA/ε/p̂** → set **CWA lamp** → **Trace ▸ Append** single-line event → export one-page Δ-report (hash tail +

consent proof). (7.21)

End of day. Recompute CSA/CWA, write **R_day**, archive `trace.jsonl` and `audit.jsonl`. (7.22)

Weekly audit. Verify **ℓ-chain** for audit logs and **h-chain** for trace; scan anomalies (out-of-role access, mass export); sign review. (7.23)

Incident ladder. S1 identifiers, S2 integrity, S3 availability; **T_detect ≤ 24 h; T_contain ≤ 48 h; T_notify (S1) ≤ 72 h**; rotate keys; re-verify chains; post-mortem. (7.24)

Retention windows. TTL_A ≤ 6 mo (IDs); **TTL_B ≤ 24 mo (content)**; **TTL_C ≤ 36 mo (derived)**; **TTL_D ≥ 36 mo (governance)**. (7.25)

De-redaction policy. Clinician/Auditor only; all de-redactions emit audited entries. (7.26)

CWA discipline. If **CSA < 0.67** or **max ε_AB > 0.05** or **$\hat{p}$ < 0.05**, flip banner to **SRA only**; no averaging, no actuation; add redundancy and refactor critics. (7.27)

7.6 "Show me" examples (ready to paste)

Two-fragment claim (redundancy).
Text note: "`S001_seg03` avoidance_high" + Sensor receipt: "bandpower window `bp_720-900`".
fragments_per_claim = 2 → vote=1 in **O3**. (7.28)

Footer of export.
`dataset_root_hash=af21…`, `session_hash_tail=…3b`, `consent=metrics_only@2025-10-18`, `CWA_OK=true`. (7.29)

7.7 Why this works (and how it maps to the math)

- **Latching** is guaranteed by the **append-only + hash-chain** ledger: once written, the event is fixed for the observer's future policy (delta-certainty). (7.30)
- **Agreement** becomes computable via **CSA/ε** with **redundant fragments**; **CWA** certifies when pooling is legal. (7.31)
- **Governance** (RBAC/consent/audit) turns those math guarantees into clinic-grade operations you can defend. (7.32)

Analogy (to remember). Your trace is a **lab EHR** where every entry is a **Git commit**; critics are your **CI tests** (CSA/ε), the **CWA lamp** is the "build passing" badge, and **Merkle roots** are your signed releases.

8. Tooling Stack & Reproducibility

8.1 The Repro Contract (what every run must record)

Environment snapshot, frozen randomness, and a small set of metrics make your results portable and auditable. We standardize this as a one-screen contract.

One-line requirements.
Environment tuple: **Env := (OS, CUDA/Driver, BLAS, Python, libs, commit).** (8.1)
Seed vector: **$\vec{s}$:= (s_py, s_np, s_torch, s_solver).** (8.2)

Budget tuple: **Bud := (wall, GPU·h, CPU·h, Wh).** (8.3)
Energy check: **Wh ≈ P̄ · wall**, measured by sampler or meter. (8.4)
Reproduce flag: **Reproduce(D, $\vec{s}$, Env) = 1** iff **|Metric′ − Metric| ≤ tol** for all registered metrics. (8.5)

Registered metrics (defaults). CSA@3, max ε_AB, CWA $\hat{p}$, $\hat{g}$, $\hat{\beta}$, $\hat{\gamma}$, $\bar{\Delta}$, ECE, κ/α, BF-curve. (8.6)

8.2 Container spec (drop-in)

Base. Ubuntu 22.04 + Python 3.11; MKL/BLAS pinned; CUDA if needed.
Locked components. CSP/ILP solver, IRL/optimizer, plotting libs; write a **lockfile** and export **tool cards** (versions + hyperparams).

One-liners to write into the image.
Container label: **io.repro.env = hash(Env).** (8.7)
Seed entrypoint: **on_start → set $\vec{s}$; export `$RUN_SEED_JSON`.** (8.8)
Budget logger: **log(Bud) per stage; attach to report footer.** (8.9)

Why containers. Same bits → same hashes → same **CSA/ε/CWA** and Δ-dials when the data are identical. The append-only trace + Merkle roots keep provenance intact.

8.3 Project scaffold (paste this tree)

```
/data/          # raw & derived (read-only in jobs)
/trace/         # trace.jsonl, graders_*.jsonl, meta.json (hashes)
/pipelines/     # csa_eps_cwa.py, delta_dials.py, picture_fit.py, irl_loop.py
/dashboards/    # panels/, report_templates/, export.py
/configs/       # *.yaml (seeds, thresholds, paths)
/tests/         # unit_*.py (idempotency, hashes, metrics)
/container/     # Dockerfile, lockfile.txt, tool_cards.json
```

Scaffold aligns with the SOP/appendix quickstarts; the same layout reproduces figures and reports with synthetic or real data.

8.4 CLI: CSA/ε/CWA + core dials (four commands)

We ship a tiny CLI so labs can compute the gates and dials without opening notebooks. Thresholds match the **CWA certificate** defaults.

(A) CSA — cross-system agreement.
*obs csa --graders /trace/graders_O.jsonl --out /dashboards/csa.json** (8.10)
Returns **CSA@3** with 7-run EMA. Target: **CSA@3 ≥ 0.67**.

(B) ε — order-sensitivity matrix.
obs epsilon --graders … --heldout /data/heldout.jsonl --out ε.json (8.11)
Check **max ε_AB ≤ 0.05** or refactor critics (pure effects, disjoint inputs).

(C) CWA — permutation test (order + phase).
obs cwa --scores /data/session_scores.json --B 1000 --out cwa.json (8.12)
Computes **T_obs** and **$\hat{p}$**; pass if **$\hat{p}$ ≥ 0.05**.

(D) Δ-dials — $\hat{g}$, $\hat{\beta}$, $\hat{\gamma}$, $\bar{\Delta}$, CUSUM.

obs delta --segments /data/segments.jsonl --out delta.json (8.13)
Reports $\hat{g}$, $\hat{\beta}$, $\hat{\gamma}$, $\bar{\Delta}$, CUSUM S_t with alarm thresholds.

Publish rule. $\text{CWA_OK} \Leftrightarrow [\text{CSA} \geq 0.67] \wedge [\max \varepsilon \leq 0.05] \wedge [\hat{p} \geq 0.05]$; otherwise "SRA only". (8.14)

8.5 Minimal unit tests (must pass before publish)

1. **Idempotent append.** Re-appending the same event **does not** change h_t. (8.15)
2. **Hash-chain integrity.** Recompute h_t; must equal stored. (8.16)
3. **CSA invariance.** Shuffling grader order leaves **CSA@3** unchanged on a stable batch. (8.17)
4. **ε sanity.** A synthetic non-commuting pair yields $\varepsilon > 0$, commuting pair $\varepsilon \approx 0$. (8.18)
5. **CWA permutation.** For IID synthetic scores, $\hat{p} \approx 0.5$ on average; for order-coupled scores, $\hat{p} \to 0$. (8.19)
6. **Seeds are real.** Running **obs delta** twice with identical $\vec{s}$ reproduces $\bar{\Delta}$ to within **tol**. (8.20)

8.6 Metric harness scripts (calibration, agreement, evidence)

We include callable scripts so every paper can re-run the same evaluations with one command.

Harness outputs (single lines).
Calibration: $\text{ECE} := \sum_b (|S_b|/N) \cdot |\text{acc}(S_b) - \text{conf}(S_b)|$. (8.21)
Agreement: $\kappa := (p_o - p_e) \div (1 - p_e), \quad \alpha := 1 - (D_o \div D_e)$. (8.22)
Evidence stream: $\Lambda_T := \sum_{t \leq T} \log \text{BF}_t$; switch if $\Lambda_T \geq \lambda$. (8.23)

CLI examples.
obs calib --probs probs.json --labels y.json --out ece.json
obs agree --labels graders.json --out kappa_alpha.json
obs bf --liks H1.json H0.json --out bf_curve.json (8.24)

8.7 Compute budget reporting (fair, apples-to-apples)

You will publish the cost of your results alongside the numbers. The harness collects budget telemetry and prints a footer block that journals can copy.

Footer fields (auto-filled).
$\text{wall} = \sum \text{stages wall_i}$; $\text{GPU·h} = \sum g_i \cdot t_i$; $\text{CPU·h} = \sum c_i \cdot t_i$; $\text{Wh} = P \cdot \text{wall}$; seeds = $\vec{s}$; $\text{env_hash} = \text{hash(Env)}$. (8.25)

Policy. Release code, configs, lockfile, and evaluation harness; include **dataset_root_hash** and **Merkle day roots** so third parties can verify end-to-end. (8.26)

8.8 One-command reproduction (Quickstart)

obs repro --config /configs/paper.yaml --export /dashboards/report.pdf (8.27)
Runs **CSA/ε/CWA**, **Δ-dials**, calibration, evidence, and emits a one-page report (numbers aligned to the compact reporting block).

What appears in the footer.
`env_hash=… seeds=… CSA@3=… max ε=… p̂=… ĝ/β̂/γ̂=… Δ̄=… ECE=… κ/α=… Λ_T=… wall/GPU·h/Wh=… dataset_root_hash=…` (8.28)

8.9 Why this stack works (and maps to our math)

- **Append-only + hashes** make writes *latch* (delta-certain in-frame) and keep provenance verifiable. (8.29)
- **CSA/ε/CWA** implement the **agreement** proxy (commuting critics + redundancy + permutation guard). (8.30)
- **Seeds + container + harness** satisfy the reproducibility checklist (same inputs → same outputs → same decisions). (8.31)
- **Budget disclosure** keeps results comparable and responsible across labs. (8.32)

Mental model. Your stack is a **flight recorder** (trace + hashes) running inside a **sealed cockpit** (container) with **three gauges** front and center (CSA bar, ε heatmap, CWA lamp). If those are green and seeds/env are locked, anyone can fly the same route tomorrow and land on the same numbers.

9. ESI "Smoothness" for Cognitive Pipelines (Optional but Handy)

9.1 What "smoothness" means here (one breath)

We borrow ESI's emulsion metaphor to keep multi-tool neuro pipelines from "curdling" (loops, contradictions, brittle order effects). Smoothness lives in a **phase region** controlled by tiny structure (**starch S**), gentle heat schedules (**decoding/decision temperature T**), and task load (**capacity–diversity K**), and it's **verified** by cross-observer agreement (CSA) and a **clump** order parameter **χ**.

Axes (single lines).
Phase axes: **T := temperature ⊕ nucleus-mass (top-p), S := % structural scaffold, K := capacity ÷ diversity.** (9.1)
Smoothness proxy: **Smooth ⇔ [χ ≤ χ*] ∧ [CSA@3 ≥ 0.67].** (9.2)

9.2 The clump order parameter χ (what you actually compute)

ESI defines **χ** as a compact, unit-free score that rises when outputs clump (loops, premature collapse, contradictions). Use it as an **early warning** alongside CSA/ε.

Definition (one line).

$\chi := w_H \cdot \Delta H\downarrow + w_L \cdot L_{loop} + w_C \cdot C_{contra}$, with $w_H+w_L+w_C=1$. (9.3)
Here **$\Delta H\downarrow$** = normalized entropy *drop* between outline→draft, **L_{loop}** = loop rate (repeats/min), **C_{contra}** = contradiction rate vs Given/Trace. Defaults: **$w_H=0.4$, $w_L=0.3$, $w_C=0.3$**. (9.4)

Alarm & action.
Alarm ⇔ $\chi \geq \chi^*$ (default $\chi^*=0.6$). → Cool T, raise S to next tier, re-run CSA/ε. (9.5)

9.3 Starch budget (tiny structure that prevents curdling)

ESI shows that **1–3%** structural tokens (or protocol scaffolds) are enough to stabilize phase without biasing content. In neuro pipelines, this is **protocol scaffolding**, not "prompting": concise slots that make checks addressable and critics commute.

Budget rule (one line).
S := {1% if v≤0.25; 2% if 0.25<v≤0.6; 3% if v>0.6}, v = task-volatility score. (9.6)

Scaffold grammar (≤120 tokens @ 4k ctx, adapt to your forms):
[Given] task & IDs • [Plan] goals→steps→tools • [Compute] succinct • [Checks] units/constraints/contradictions • [Trace] cite fragment IDs • [Answer] single final. (9.7)

Why S works (free-energy view, plain). Bounded structure slightly **raises early conditional entropy** (prevents premature collapse) while keeping macro-shape addressable; this reduces loop attractors. **Result:** χ↓, CSA↑. (Proof sketch in ESI §6.)

9.4 Sous-vide schedules (gentle heat across passes)

Run **cool → warm → cool** decoding/decision passes so the system explores, commits, then verifies—mirroring outline→draft→check. Works for LLM toolcalls and for analysis/label-fusion passes in your lab stack.

Schedule (one line).
T_{pass} := { T_1=cool (temp 0.3–0.5 / top-p 0.8–0.9), T_2=warm (0.6–0.8 / 0.9–0.98), T_3=cool (0.2–0.4 / 0.7–0.9) }. (9.8)

Operational rule.
If χ↑ or ε hot (>0.05), drop to T_1 and raise S tier; re-test CSA/ε before committing labels. (9.9)

9.5 How ESI plugs into CSA/CWA (the verification link)

ESI equates "smoothness" with **agreement**: commuting critics + redundant traces → order-insensitive majorities. Keep the **CWA certificate** as the *legal gate* for pooling/actuation; use **χ** as a *stability meter* that tells you when to cool/structure before you even get to CWA.

Gate (recall).
CWA_OK ⇔ [CSA@3 ≥ 0.67] ∧ [max ε_{AB} ≤ 0.05] ∧ [$\hat{p}$ ≥ 0.05]. (9.10)

Two-light rule.
Publish/act only if (CWA_OK = green) ∧ ($\chi \leq \chi^*$). (9.11)

9.6 Neuro-pipeline translation (drop-in settings)

Where "S" lives. Protocol cards and analysis templates, not free text:
[TaskSpec], [Windows], [Units], [Invariants], [TraceIDs], [Answer]. (9.12)

Where "T" lives. Multi-pass analysis:
Pass-1 (explore features, cool), Pass-2 (fit/segment calls, warm), Pass-3 (verify/CSA, cool). (9.13)

Where "χ/CSA" live. χ panel from draft logits/labels + loop/contra rates; CSA/ε from your three pure critics (Units • NLI-vs-Given • Trace-referencer). (9.14)

9.7 Minimal recipes (paste-and-run)

(A) Volatility → starch tier.
$v := \alpha_{len} \cdot v_{len} + \alpha_{sym} \cdot v_{sym} + \alpha_{amb} \cdot v_{amb} + \alpha_{hist} \cdot v_{hist}$; pick S by (9.6). (9.15)

(B) χ monitor (per batch).
$\chi_t := 0.4 \cdot \Delta H\downarrow_t + 0.3 \cdot L_{loop,t} + 0.3 \cdot C_{contra,t}$; Alarm if $\bar{\chi} \geq 0.6$. (9.16)

(C) Heat controller.
if $\bar{\chi} \geq \chi^*$ or max $\varepsilon > 0.05$ → $T := T_1$; S:=next tier; else keep $T := T_2$ then T_3 for verify. (9.17)

(D) Commit rule.
Commit labels ⇔ CWA_OK ∧ $\chi \leq \chi^*$. Else: SRA only; add redundancy; retry. (9.18)

9.8 Defaults & dashboards (one screen)

Defaults. S ∈ {1,2,3}%, **$\chi^* = 0.6$**, **T_pass** as (9.8), **CSA/ε** thresholds per §2, CWA α=0.05. (9.19)

Dashboard rows.
Top: **CSA@3** trend; Mid-L: **ε** heatmap; Mid-R: **χ** sparkline + alarm; Bottom: **CWA lamp** + action coach ("Cool T / Raise S / Add redundancy / SRA only"). (9.20)

9.9 Why this is principled (not just vibes)

ESI's "starch + sous-vide" maps exactly to the observer math: bounded scaffolds make the policy **measurable** and critics **approximately commute**; redundant trace fragments meet the SBS condition; CSA/CWA certify objectivity. The **free-energy** sketch explains why small S lowers loop channels (χ↓). That's why the same settings that make LLM tool use creamy also stabilize EEG/MEG/fMRI/ECoG/TMS analysis stacks.

Analogy. Keep your **sauce** from curdling: add a **pinch of starch** (S), warm it **slowly** (T), don't overload the pot (K), and **taste-test** with three tasters who agree no matter the order (CSA). If the sauce starts to clump (χ↑), cool it, add a touch more starch, and only plate when the **CWA** light is green.

10. Metrics & Dashboards (What to Look At Every Day)

10.1 Core KPIs (single-line, Blogger-ready)

Cross-system agreement (three independent, non-mutating critics):
CSA@3 := (1/N) · Σ_j 1{ majority label for item j unchanged under all grader orderings }. (10.1)

Order-sensitivity (held-out set D):
ε_{AB} := $Pr_{d\sim D}$[A∘B(d) ≠ B∘A(d)]. (10.2)

Redundancy (receipts per claim):
ρ := (total fragments logged) ÷ (total claims). (10.3)

Stability dial (segment-level; see §4):
$\Delta_t := \hat{g}_t \cdot \hat{\beta}_t - \hat{\gamma}_t$, $\bar{\Delta} := EMA_W(\Delta_t)$. (10.4)

CWA certificate (pooling legality):
CWA_OK ⇔ [CSA@3 ≥ 0.67] ∧ [max ε_{AB} ≤ 0.05] ∧ [$\hat{p}$ ≥ 0.05]. (10.5)

Why these five? They're the computable proxies of the observer-with-trace guarantees: latching (delta-certainty), commuting effects, and redundant records → objectivity. The dashboard bundles them into one glance.

10.2 Threshold bands (defaults you can ship)

- **CSA@3:** green ≥ **0.67**, amber 0.50–0.67, red < 0.50. (10.6)
- **ε_{AB} (each pair):** green ≤ **0.05**, amber 0.05–0.08, red > 0.08. (10.7)
- **ρ (redundancy):** green ≥ **2.0**, amber 1.5–2.0, red < 1.5. (10.8)
- **$\bar{\Delta}$ (needle):** green ≤ **−0.2**, amber −0.2…+0.2, red ≥ **+0.2**. (10.9)
- **CWA light: green** iff (10.5); else **red** → **SRA only**. (10.10)

10.3 Screen layout (copy this arrangement)

Top-left: $\bar{\Delta}$ big needle.
Top-right: **CUSUM S_t** with alarm line **h**.
Middle-left: $\hat{g}$, $\hat{\beta}$, $\hat{\gamma}$ mini-gauges (arrows vs. previous session).
Middle-right: **CSA@3 bar + ε heatmap + ρ**.
Bottom: **CWA light** and **Action Coach** (if $\bar{\Delta}$ red: suggest "reduce g / chunk β / add γ"; if CWA red: "SRA only; add redundancy; refactor critics").

Analogy. Think cockpit: the **Δ needle** is your horizon, the **CSA bar + ε map** are your instrument cross-checks, and the **CWA lamp** is the "clear to land" light.

10.4 Ready-to-paste plotting calls (minimal stubs)

CSA trend (with band):

```
plot_line(csa_series, y_ref=0.67, title="CSA@3 (goal ≥ 0.67)")
```
(10.11)

ε heatmap (flag hot cells):

`plot_heatmap(epsilon_matrix, vmax=0.10, annotate_hot>0.05)` (10.12)

Redundancy bar:
`plot_hbar({"fragments/claim": rho}, bands=[1.5, 2.0])` (10.13)

Δ needle + CUSUM:
`plot_gauge(delta_bar, bands=[-0.2, 0.2]); plot_line(S_t, y_ref=h)` (10.14)

CWA lamp:
`lamp = "green" if (csa≥0.67 and max_eps≤0.05 and p_hat≥0.05) else "red"` (10.15)

> These stubs match the CLI from §8 (`obs csa`, `obs epsilon`, `obs cwa`, `obs delta`) so you can wire them without notebooks.

10.5 Alert rules (paste into your ops playbook)

A1. Quarantine noisy critic.
If **ε_AB spikes above 0.05** for any pair → **quarantine that critic**, split inputs (make it pure), and re-run ε/CSA before publishing. (10.16)

A2. Cool guidance on CSA dip.
If **CSA@3 EMA < 0.67 → cool $\hat{g}$** (reduce reframe rate), raise redundancy ρ, and hold pooling (CWA=red). (10.17)

A3. Permutation fail.
If **$\hat{p}$ < 0.05** (order/phase) → **SRA only**; stabilize timing; do not average or actuate. (10.18)

A4. $\bar{\Delta}$ in red.
If **$\bar{\Delta} \geq +0.2$** → Action Coach suggests **(g↓ / β↓ / γ↑)**, then re-check CSA/ε before committing new labels to Trace. (10.19)

A5. Verify before export.
Publish only if **VerifyTrace=1** and **CWA_OK=green**; attach `dataset_root_hash` in footer. (10.20)

10.6 Daily loop (10 minutes, every session)

1. Compute **$\hat{g}$, $\hat{\beta}$, $\hat{\gamma}$, $\bar{\Delta}$, S_t**; 2) Run **CSA/ε**; 3) Run **CWA $\hat{p}$**; 4) Render the **five-line board**; 5) Append event to **Trace** (hash-chained); 6) Export one-page report (numbers aligned in footer). (10.21)

Footer block (auto).
`CSA@3=… max ε=… ρ=… p̂=… ĝ/β̂/γ̂=… Δ̄=… S_max/h=… CWA=… dataset_root_hash=…` (10.22)

10.7 Why this dashboard is defensible

- **Math-to-ops link.** Latching (fixedness) + commuting effects + redundancy ⇒ objectivity; your **CSA/ε/CWA** panels are the operational mirrors. (10.23)
- **Clinically legible.** One dial (**$\bar{\Delta}$**), one lamp (**CWA**), one cross-check (**CSA + ε + ρ**)—the same recipe used in our closed-loop control and Freud→Control bridges. (10.24)

- **Reproducible.** The ObserverOps blueprint ships the SDK, dashboards, and config playbooks so another lab can rerun your exact board. (10.25)

Bottom line. If the **Δ needle** settles, the **CSA bar** stays high, the **ε map** stays cool, redundancy **ρ ≥ 2**, and the **CWA light** is green—you can trust your averages and (if applicable) arm interventions. If any lamp flips red, drop to **SRA only**, fix the culprit, and try again.

11. Failure Modes & Falsification

11.1 Quick map of what can break (tests you can run)

Non-measurable policy (Π not a function of the written past).
Measurability pass: **Meas_OK ⇔ Var[Π(T_t, y_t, c_t) | T_t fixed] ≤ θ_m.** (11.1)
Replay the same **T_t** (and context **y_t,c_t**) through Π; if actions vary, Π is peeking at hidden/unstated state → stop pooling/actuation. This violates the "policy is $\mathscr{F}$_t-measurable" premise behind latching.

Non-commuting checks (order changes the verdict).
Commutation pass: **Commute_OK ⇔ max ε_AB ≤ 0.05**, where **ε_AB := Pr[A∘B ≠ B∘A]**. (11.2)
If any ε cell is hot, critics interact; fix inputs or split effects before averaging.

Missing redundancy (no receipts).
Redundancy pass: **ρ := fragments/claim ≥ 2.0** (text + time-locked sensor). (11.3)
Without SBS-style redundancy, agreement cannot "stick."

Frame mis-alignment (bad remaps across areas/modalities).
Alignment pass (estimate $\hat{f}$ by Procrustes/CCA): **E_align := ‖ $\hat{f}$(X_A) − X_B ‖_F ÷ ‖X_B‖_F ≤ 0.2.** (11.4)
High **E_align** means compare/average *within* frame only (SRA).

Pooling legality (recap).
CWA_OK ⇔ [CSA@3 ≥ 0.67] ∧ [max ε_AB ≤ 0.05] ∧ [$\hat{p}$ ≥ 0.05]. (11.5)
If false → SRA only, fix the cause, and re-run checks.

Analogy. If your thermometers fight each other (ε hot) or you lost the receipts (ρ low), don't average them; read each thermometer separately (SRA).

11.2 Falsification, not just "error-bars" (how to disprove us fairly)

(F1) Latching-in-practice test.
We predict: writing **e_t** changes action distributions because Π reads **T_t**.
Test: **Δ_u := d_TV[u | (T⊕e) , u | (T⊕e′)]** over matched sessions; **expect Δ_u > θ_l**. (11.6)
If **Δ_u ≈ 0** *after verifying Meas_OK*, implementation contradicts "collapse-as-conditioning" at the policy layer—report as a falsifier.

(F2) AB-fixedness test under stated premises.
Premises satisfied: **Commute_OK ∧ ρ ≥ 2** on a held-out set. Prediction: **order-invariant agreement**.
Falsifier: **∃d : A∘B(d) = B∘A(d) ∧ redundant writes, yet majority label flips under order**. (11.7)

If observed, this violates the AB-fixedness guarantee (not just low power). Publish as a counterexample.

(F3) Permutation invariance after CWA pass.
Premise satisfied: **CWA_OK = green**. Prediction: $\hat{\mathbf{p}}$ **≈ uniform ≥ 0.05** under order/phase shuffles.
Falsifier: $\hat{\mathbf{p}}$ **≪ 0.05** repeatedly with green gates → hidden coupling or frame leak; if audits clear, the CWA recipe is insufficient—document and amend thresholds. (11.8)

(F4) Hinge switching without evidence.
Prediction (hinges): no switch unless log-BF crosses λ.
Falsifier: **Λ_T < λ yet hinge flipped** across raters/tools passing CSA → hyperprior spec invalid. (11.9)

11.3 Stress tests (break it on purpose before it breaks you)

ST-1: Non-measurable Π. Replay identical **T_t** but hide a private cache Π was using; flag **Meas_OK** if actions now differ. Expect (11.1) to fail; fix by making inputs explicit in **T**.

ST-2: Non-commuting critics. Let **O2** read a post-edited draft from **O1**; ε spikes by design; verify (11.2) catches it; refactor to *pure* critics with disjoint inputs.

ST-3: Missing redundancy. Drop sensor receipts (ρ→1); CSA becomes brittle; CWA flips red. Restore dual fragments to pass (11.3), (11.5).

ST-4: Frame scramble. Insert a random orthogonal map between ROI sets; **E_align**↑, CSA falls across frames; verify (11.4) forces per-frame SRA.

ST-5: Hinge jitter. Force frequent hinge flips below λ; BF stream alarms (11.9) should fire; if not, calibrate λ or BF estimators.

11.4 Counterexample library (template)

```
{
  "id": "CE_ST2_noncommute_v1",
  "purpose": "Show ε hot when critics interact",
  "setup": {
    "O1": "units/invariants",
    "O2": "NLI reading O1-mutated text (bad)",
    "O3": "trace referencer"
  },
  "expected": {
    "max_epsilon": ">0.05",
    "CSA@3": "<0.67",
    "CWA_OK": false
  },
  "fix": "Make O2 read Given/Trace only; split O2 if needed"
}
```

Ship a dozen such JSON cases (non-measurable Π, missing ρ, frame scramble, hinge jitter) and keep them in `/tests/counterexamples/`.

11.5 When to revert to *per-case* (SRA) immediately

SRA_NOW ⇔ [¬Meas_OK] ∨ [max ε_AB > 0.05] ∨ [ρ < 2] ∨ [E_align > 0.2] ∨ [p̂ < 0.05]. (11.10)
Turn off pooling and actuation; append an incident note; fix, re-verify, then restore CWA flow.

11.6 Ops glue (tiny scripts you'll actually run)

Policy measurability probe.
δΠ := mean_t 1{ Π(T_t,y_t,c_t) ≠ Π(T_t,y_t,c_t) on exact replay }. (11.11)
Pass ⇔ δΠ ≤ θ_m (e.g., 1%).

Alignment probe (per batch).
E_align := ‖ f̂(X_A) − X_B ‖_F ÷ ‖X_B‖_F; Pass ⇔ E_align ≤ 0.2. (11.12)

Falsifier logger.
Append `{ test_id, premise_checks, metric_values, verdict }` to `trace.jsonl`; hash-chain as usual so falsifiers are auditable facts, not opinions. (11.13)

11.7 Why this is the right kind of "hard to fake"

- **Latching & measurability.** If Π ignores **T**, replay exposes it (11.1); the algebraic fixed-point view explains why this matters.
- **Agreement from commuting + redundancy.** ε and ρ are the computable guards for AB-fixedness; our CWA gate formalizes pooling legality.
- **Hinges with evidence.** Bayes-factor streams make "certainty" falsifiable by design.
- **Governance spine.** Hash-chained traces, RBAC/consent, and incident runbooks make failures visible and recoverable.

Plain recap. If your "thermometers" (critics) interfere, or you lack receipts, or you're comparing apples in one frame to oranges in another, **don't average**. Use these tests to catch each failure fast, publish per-case when needed, and—crucially—keep a small library of counterexamples so you can prove your gates actually work. That's falsifiable science, not vibes.

12. Case Studies & Starter Protocols

12.1 EEG microstate labeling — order-swap critics + redundant traces

Goal. Label EEG segments with microstates, prove objectivity (CSA/ε), and only pool sessions when the **CWA** gate is green.

Neural proxy dials. β_neuro := state-switch rate (HMM/microstates) per minute; γ_neuro := 1 ÷ τ_recover; Δ_neuro := g_neuro·β_neuro − γ_neuro. (12.1)

Critics (pure, non-mutating).
O₁=Units/constraints (windowing, SNR, artifact bounds); O₂=Microstate/HMM labeler; O₃=Trace-referencer (verifies ≥2 receipts/claim). **CSA_neuro := fraction stable under all orderings; ε_AB := Pr[A∘B ≠ B∘A].** (12.2)

Redundancy rule. Every claim = **text note + time-locked sensor fragment** (≥2 receipts)

in an append-only trace with hash-chain: **$h_0:=0$; $h_t:=H(h_{t-1} \| \text{canonical_json}(e_t))$.** (12.3)

CWA gate (pooling legality). $\text{CWA_neuro} \Leftrightarrow [\text{CSA@3} \geq 0.67] \wedge [\max \varepsilon_{AB} \leq 0.05] \wedge [\hat{p} \geq 0.05]$ (order/phase permutations). (12.4)

Pre-registered endpoints.
Primary: **CSA@3**, **max ε_AB**, **$\hat{p}$**; Secondary: **β_neuro**, **γ_neuro**, **$\bar{\Delta}$_neuro** (EMA). **Pass $\Leftrightarrow$ CWA_neuro=green.** (12.5)

SOP (copy-paste).

1. Segment EEG → compute features; 2) Run $O_1/O_2/O_3$; 3) Compute **ε**, **CSA@3**; 4) Permute order + circular phase for **$\hat{p}$**; 5) If **CWA=green**, average; else **SRA only**; 6) Append one event to **trace.jsonl** with **prev_hash/hash**. (12.6)

One-liners to report.
$\beta_{neuro} = \text{transitions/min}$; $\gamma_{neuro} = 1 \div \tau_{recover}$; $\bar{\Delta}_{neuro} = \text{EMA}(\hat{g} \cdot \hat{\beta} - \hat{\gamma})$. (12.7)

Why it's defensible. Order-insensitive majors + redundant receipts realize the agreement theorem; the hash-chain makes latching operational.

12.2 fMRI/TMS: PFC-as-meta-observer perturbation (CWA-gated)

Goal. Causally test the **g_neuro** handle by perturbing dlPFC and observing Δ_neuro.

Neural handles. $g_{neuro} :=$ strength(PFC→target) (DCM/Granger or TMS-evoked transfer); **$\Delta_{neuro} := g_{neuro} \cdot \beta_{neuro} - \gamma_{neuro}$.** (12.8)

Design (Battery A). Single-pulse **dlPFC TMS** blocks interleaved with baseline; measure evoked transfer + ROI patterns; parallel EEG/MEG for **β_neuro**, probes for **γ_neuro**.
Prediction: lowering **g_neuro** (TMS) → **$\Delta_{neuro}\downarrow$**, hitting-time (CUSUM) shifts right. (12.9)

Objectivity checks (before pooling). Run **CSA_neuro** across {EEG critic, MEG critic, fMRI critic}; compute **ε** and permutation **$\hat{p}$**; require **CWA_neuro=green**. (12.10)

Pre-registered endpoints.
Primary: **Δ_{neuro} change** (pre vs post TMS), **CUSUM S_t** shift; Secondary: **g_neuro** gain drop, **CI** (optional binding cascade). **Fail if Δ_{neuro} unchanged with verified $g_{neuro}\downarrow$.** (12.11)

SOP (copy-paste).

1. Acquire fMRI (or SEEG where available) + TMS timing logs; 2) Estimate **g_neuro**; 3) Run EEG/MEG microstates for **β_neuro**, probes for **γ_neuro**; 4) Compute **$\bar{\Delta}$_neuro**, **S_t**; 5) Gate pooling by **CWA**; 6) Append hashed events; 7) Export one-page report with dataset root hash. (12.12)

Why it's defensible. "Meta-observer PFC" is tested via directed gain; Δ-dial gives a one-number causal readout; CWA avoids order/phase illusions.

12.3 ECoG language-game task — hinge switching metrics

Goal. Operationalize “meaning = use” and **hinge certainty** in an intracranial language-game, reporting calibration and BF trajectories.

Meaning-as-use (policy). Estimate $\pi_\theta(a|x) \propto \exp(\theta \cdot \varphi(x,a))$ from interaction/choice logs; report **ΔU** versus baseline and **ECE** for calibration. (12.13)

Hinges (slow hyperpriors, BF stop). Maintain $\Lambda_T := \sum_{t \le T} \log BF_t$; **switch hinge iff** $\Lambda_T \ge \lambda$ (cost-of-change). (12.14)

Agreement & publicness. Report **κ/α** across raters; pre-commit CWA gate before any across-trial pooling. (12.15)

Pre-registered endpoints.
Primary: **Fit AUC** (picture→world CSP), **ΔU** (policy lift), **ECE** (↓ better), **κ/α** (↑ better), **BF trajectory** Λ_T (hinge switch events). (12.16)

SOP (copy-paste).

1. Build scene-graphs (CSP) for “picture fit”; 2) Run max-ent IRL for π_θ; 3) Compute **ECE**, **κ/α**; 4) Maintain BF streams for hinges; 5) Gate pooled metrics by **CWA**; 6) Append hashed trace events with tool cards/versions. (12.17)

Why it’s defensible. Wittgenstein-operationalized gives CSP/IRL/BF instruments + reporting standards (prereg, artifacts, thresholds).

12.4 Pre-registration mini-template (fill & paste)

- **Hypotheses.** H_1: TMS lowers **g_neuro** and thus **Δ_neuro**; H_2: microstate **β_neuro** predicts $\bar{\Delta}$; H_3: hinge switches occur only when $\Lambda_T \ge \lambda$. (12.18)
- **Primary endpoints. CSA@3, max ε_AB, $\hat{p}$; $\bar{\Delta}$_neuro**; **Fit AUC, ΔU, ECE, κ/α, Λ_T**. (12.19)
- **Stopping rules. CWA=red ⇒ SRA only;** hinge switch at $\tau^* = \inf\{T : \Lambda_T > \lambda\}$. (12.20)
- **Artifacts.** Code, configs, seeds, container lockfile, dataset/Merkle roots, tool cards. (12.21)

12.5 Copy-paste SOP blocks (per case)

EEG microstates (CSA/ε/CWA).
Compute: β_neuro, γ_neuro, $\bar{\Delta}$_neuro, CSA, ε, $\hat{p}$. Publish: only if **CWA=green**; else **SRA**. **Trace:** append JSONL event with **prev_hash/hash**. (12.22)

fMRI/TMS (Δ perturbation).
Estimate: g_neuro (DCM/evoked transfer); recompute **$\bar{\Delta}$_neuro**; **CUSUM S_t** shift. **Gate: CWA**. **Fail:** Δ unchanged with g drop. (12.23)

ECoG language-game (hinges).
Run: CSP fit → IRL policy → ECE/κ/α → BF stream. **Switch:** only when $\Lambda_T \ge \lambda$. **Gate**

pooled AUC/ΔU by CWA. (12.24)

12.6 Compact reporting footer (paste under each figure)

`CSA@3=… max ε=… ρ̂=… ρ=… β_neuro=… γ_neuro=… Δ̄_neuro=… g_neuro=… S_max/h=… FitAUC=… ΔU=… ECE=… κ/α=… Λ_T=… env_hash=… seeds=… dataset_root_hash=…` (12.25)

12.7 Why these three are "starter-ready"

- **Same math, three modalities. $y=\Omega\hat{}[x]$, $g/\beta/\gamma\to\Delta$**, CSA/ε/CWA, and hashed traces are shared scaffolds; only the sensors change. (12.26)
- **Causal + calibrated.** PFC perturbation tests **g_neuro**; microstates probe **β_neuro**; ECoG games quantify **meaning** and **hinges** with **BF** and calibration (ECE). (12.27)
- **Repro-first.** Prereg, seeds, containers, and Merkle roots make results portable; the CWA gate keeps averages honest. (12.28)

Analogy (to brief the team). Three different kitchens, same **line check**: do the tasters agree regardless of order (CSA/ε)? Is the house special stable (Δ needle)? Are receipts in the drawer (redundancy + hashes)? If yes, serve the average; if not, plate **per case**.

13. Ethics, Safety, and Governance

13.1 What we enforce (one screen, one-liners)

De-identification (default-on).
Redaction rule: **redact(text) := Redact(text; rules R, token set Σ).** (13.1)
Salted tokenization (no plaintext IDs): **map[token] := H_salt(identifier).** (13.2)
Retention windows: **TTL_A≤6 mo, TTL_B≤24 mo, TTL_C≤36 mo, TTL_D≥36 mo.** (13.3)

Consent & cognitive liberty.
Consent machine-state: **consent(u,scope) ∈ {none, minimal, metrics_only, full_sharing}.** (13.4)
Off-switch: **if consent < metrics_only ⇒ stop metrics & exports.** (13.5)

Auditability (tamper-evident).
Trace hash-chain: **h₀:=0; h_t:=H(h_{t−1} ∥ canonical_json(e_t)).** (13.6)
Verify trace: **VerifyTrace(T)=1 ⇔ recompute(h_T)=stored(h_T).** (13.7)
Admin/audit log chain: **ℓ₀:=0; ℓ_t:=H(ℓ_{t−1} ∥ canonical_json(log_t)).** (13.8)

Role-based access (RBAC).
Allow rule: **Allow(u, a, r)=1 ⇔ [role(u)∈policy[a,r] ∧ consent(r)≥level(a)].** (13.9)

"Do-not-average" locks (CWA gate).
Publish/act only if: **CWA_OK ⇔ [CSA@3≥0.67] ∧ [max ε_AB≤0.05] ∧ [$\hat{p}$≥0.05].** (13.10)
UI lock: **DoNotAverage = 1{¬CWA_OK}.** (13.11)

Sharing guardrail.

PublishOK ⇔ [CWA_OK] ∧ [k-anonymity k≥5] ∧ [no Class-A fields]. (13.12)

Incident clock (S1/S2/S3).
T_detect≤24 h; T_contain≤48 h; T_notify(S1)≤72 h. (13.13)

Drift & fairness monitors.
Drift: **D_t:=PSI(P_t,P_ref) (or KL); alert if D_t≥δ.** (13.14)
Parity: **gap_CSA := max_group(CSA) − min_group(CSA); flag if gap_CSA>0.15.** (13.15)

Human-in-the-loop.
"Dashboard is assistive; clinician decides. No covert collection. Pooling legal only with CWA_OK; else SRA (per-case)."

Why these are principled. Latching and agreement are made operational by **append-only hashes** and **CWA gating**; RBAC/consent enforces cognitive liberty; all of this ties back to the formal observer calculus (delta-certainty; commuting effects).

Analogy. *Seatbelts and a black box:* you always wear the seatbelt (redaction/RBAC), the cockpit recorder is immutable (hash-chains), and you **don't take off** unless the tower's green light (CWA) is on.

13.2 Copy-paste policy blocks (minimal, production-ready)

A) Access & consent (RBAC).

```
{
  "rbac": {
    "roles": ["patient","clinician","annotator","supervisor","ops","researcher","auditor"],
    "allow": "Allow(u,a,r)= (role(u)∈policy[a,r]) && (consent(r)≥level(a))",
    "reviews": {"cadence_days":30, "revoke_inactive_days":60}
  },
  "consent_states": ["none","minimal","metrics_only","full_sharing"],
  "off_switch": "if consent<metrics_only: stop_metrics=true; stop_exports=true"
}
```

(Ready-to-use mappings and consent semantics per Appendix F.)

B) Exports & residency.

```
exports:
  allowlist_targets: ["grc-vault","soc-binder"]
  format: "jsonl+sig"
  dlp: { enabled: true, block_on_match: ["P3","auth_token"] }
  residency: { eu: ["eu-grc-vault"], us: ["us-grc-vault"] }
  rate_limits: { max_bytes_s: 2000000, max_records_min: 5000 }
```

(Ship signed manifests; block non-allowlisted sinks; enforce DLP and residency.)

C) Do-not-average lock (UI).

```
{
  "cwa_gate": "CWA_OK = (CSA>=0.67) && (max_eps<=0.05) && (p_hat>=0.05)",
  "ui": { "avg_button_enabled": "CWA_OK", "banner_if_false": "SRA only" }
}
```

(Prevents pooled reports/actions when evidence is unstable.)

D) Audit log schema (hash-chained).

```
{
  "log_id":"L000123",
  "iso_time":"2025-10-18T16:03:22Z",
  "actor_id":"u_451","actor_role":"Clinician",
  "action":"READ","resource":"segments.jsonl","resource_id":"S001_seg03",
  "details":{"fields":["r_strength","stance_shift"]},
  "ip":"203.0.113.45","sig":"ed25519:...=="
}
```

Audit chain: **ℓ_t = H(ℓ_{t−1} ∥ canonical_json(log_t))**; verify with **VerifyAudit(L)=1**. (13.16)

13.3 De-identification SOP (default-on)

1. **Classify fields** into **A (identifiers)**, **B (content)**, **C (derived)**, **D (governance)**; redact A; tokenize references in B; keep C/D separate.
2. **Token map** stored in Ops-only vault; de-redaction allowed only for Clinician/Auditor, always audit-logged.
3. **Right-to-erasure** via crypto-erase (rotate keys; rebuild de-ID view) while preserving minimal anchored hashes for integrity.

13.4 Human oversight & ethical use

- **Humans decide.** The board is advisory; sensitive interventions require **clinician approval**; researchers receive **de-ID aggregates only** with CWA_OK.
- **Hinge transparency.** Declare hinge priors, Bayes-factor thresholds, and who set them (Hinge Card); publish subgroup calibration and agreement.
- **Risk limits.** Cap framing intensity/frequency; cool-off windows; full debrief for any deception-style task; pre-checks for harm content.

13.5 Security & compliance quicksheet

- **Crypto.** AES-256 at rest; mTLS in transit; keys in KMS/HSM; split-role approvals for key export.
- **Privacy dashboards.** PII detections, encryption coverage, export DLP blocks, RBAC drift, residency violations, trace integrity.
- **SOC/ISO mapping.** RBAC reviews, signed admin logs, export allowlists, privacy-lint in CI; include **policy hash** and **ledger head hash** in every export.

13.6 Failure response (who does what, when)

Severity ladder: **S1 identifiers, S2 integrity, S3 availability**. Clock in plain text: **detect ≤24 h; contain ≤48 h; notify (S1) ≤72 h**. (13.17)
Recipe: **isolate → rotate keys → VerifyTrace & VerifyAudit → notify → root-cause →**

harden → post-mortem (hash the report).

13.7 Why this governance maps to the math (one breath)

- **Latching ⇒ immutability:** Append-only, hash-chained traces make "you can't unhappen a write" operational (delta-certainty).
- **Agreement ⇒ CWA gate:** Commuting critics + redundancy + permutation guard yields a computable legality test for averaging (CWA_OK).
- **Cognitive liberty ⇒ RBAC/consent:** The person controls scope; researchers get aggregates only under **PublishOK**.
- **Falsifiable ops:** All accesses and exports are signed and chain-verified; drift/fairness meters expose bias over time.

Bottom line. Treat privacy, consent, and audit as **first-class invariants**. Keep the **CWA lamp** wired to a **do-not-average** lock. If evidence or consent drops, the system **falls back to SRA** automatically. That's how rigorous math (latching, agreement) becomes everyday clinical safety you can defend—to patients, IRBs, and auditors alike.

14. Appendices — A. Notation & One-Line Identities (Blogger-ready Unicode)

> Paste as-is. All equations are single-line Unicode with (14.n) tags; symbols match Sections 0–13.

A1. Core objects & symbols

Observer triplet (measure, write, policy): $\mathcal{O} := (M, W, \Pi)$. (14.1)
Append-only trace: $T_t = T_{t-1} \oplus e_t$. (14.2)
Event schema (minimal): $e_t := (\tau_t, \text{channel}_t, \text{label}_t, \text{meta}_t)$. (14.3)
Hash chain (tamper-evident): $h_0 := 0;\ h_t := H(h_{t-1} \Vert \text{canonical_json}(e_t))$. (14.4)
Trace verification: $\text{VerifyTrace}(T)=1 \Leftrightarrow \text{recompute}(h_T)=\text{stored}(h_T)$. (14.5)

Policy call (controller reads record): $u_t = \Pi(T_t, y_t, c_t)$. (14.6)
World + readout (no model commit): $x_{t+1} = F(x_t, u_t, T_t) + \eta_t;\ y_t = H(x_t) + \varepsilon_t$. (14.7)

Latching (delta-certainty of own write): $\Pr(e_t \mid T_t)=1;\ \mathbb{E}[1\{e_t=a\} \mid T_t]=1\{a=e_t\}$. (14.8)
Commutation (order-insensitive check): $A \circ B(d)=B \circ A(d)$. (14.9)

CSA (3 critics, order-invariant majority): $\text{CSA@3} := (1/N)\cdot\Sigma_j\ 1\{\text{majority label unchanged under all orderings}\}$. (14.10)
Order-sensitivity (held-out D): $\varepsilon_{AB} := \Pr_{d\sim D}[A \circ B(d) \neq B \circ A(d)]$. (14.11)
Permutation p-value (order+phase): $\hat{p} := (1/B)\cdot\Sigma_{b=1}^{B}\ 1\{|\text{mean}(s)-\text{mean}_{\pi_b}(s)| \geq |\text{mean}(s)-\text{mean}_{\pi}(s)|\}$. (14.12)
CWA gate (pooling legality): $\text{CWA_OK} \Leftrightarrow [\text{CSA@3} \geq 0.67] \wedge [\max \varepsilon_{AB} \leq 0.05] \wedge [\hat{p} \geq 0.05]$. (14.13)

Stability dial (behavior/physio): **Δ := ĝ·β̂ − γ̂.** (14.14)
Guidance slope: **ĝ := cov(r,s) ÷ var(r).** (14.15)
Amplification rate: **β̂ := (1/T)·Σ_t (jumps_t ÷ minutes_t).** (14.16)
Damping buffer: **γ̂ := 1 ÷ τ_recover.** (14.17)

Neuro mirrors: **Δ_neuro := g_neuro·β_neuro − γ_neuro; β_neuro := transitions/min; γ_neuro := 1 ÷ τ_recover(neural).** (14.18)
Directed gain (meta-observer PFC): **g_neuro := strength(PFC→target) (DCM/Granger/evoked).** (14.19)
Binding cascade: **CI := coh(V1↔IT)·coh(IT↔PFC)** (lag-ordered). (14.20)
Claustral synchrony: **CL_sync := mean_coh(claustrum↔cortex)_{integrated} − baseline.** (14.21)

Picture→world fit (CSP homomorphism): **∃h:V→D s.t. (v1,…,v_m)∈Q_j ⇒ (h(v1),…,h(v_m))∈R_j.** (14.22)
Meaning = use (max-ent policy): **π_θ(a|x) ∝ exp(θ·φ(x,a)).** (14.23)
Calibration (ECE, b bins): **ECE := Σ_{b=1}^B (|S_b|/N)·|acc(S_b)−conf(S_b)|.** (14.24)
Agreement (κ, α): **κ := (p_o−p_e)÷(1−p_e); α := 1 − (D_o ÷ D_e).** (14.25)

Hinges (Bayes-factor stream): **Λ_T := Σ_{t≤T} log BF_t; switch hinge iff Λ_T ≥ λ.** (14.26)

Frame map & alignment error: **E_align := ‖f̂(X_A)−X_B‖_F ÷ ‖X_B‖_F.** (14.27)

ESI phase knobs: **S := starch %; T := decoding/decision temperature; χ := clump order parameter.** (14.28)
Clump score (default weights): **χ := 0.4·ΔH↓ + 0.3·L_loop + 0.3·C_contra.** (14.29)

Psychoanalytic operators (scheduleable):
Repression: **Ŝ_tight[x] := Ŝ[x] + Λ·χ_taboo(x).** (14.30)
Isolation: **U_⊥(C) := diag(C).** (14.31)
Projection: **x′ := R_θ·x.** (14.32)
Sublimation: **V_id^→ := V_id + μ·B_safe; −∇V_id^→ = −∇V_id − μ∇B_safe.** (14.33)

PID (discrete): *e_t=Δ−Δ_t; z_t=z_{t−1}+e_t·Δt; u_t=K_p e_t + K_i z_t + K_d (e_t−e_{t−1})/Δt.** (14.34)
LQR (local, discrete): **u_t=−Kx_t; K=dlqr(A,B,Q,R) with x_{t+1}≈Ax_t+Bu_t.** (14.35)

A2. Identities, passes, and "red→green" fixes

Policy measurability pass: **Meas_OK ⇔ Var[Π(T_t,y_t,c_t) | T_t,y_t,c_t fixed] ≤ θ_m.** (14.36)
Commutation pass: **Commute_OK ⇔ max ε_AB ≤ 0.05.** (14.37)
Redundancy pass: **ρ := fragments/claim ≥ 2.0.** (14.38)
Alignment pass: **Align_OK ⇔ E_align ≤ 0.2.** (14.39)

Publish/act rule: **PublishOK ⇔ CWA_OK ∧ VerifyTrace(T)=1 ∧ consent ≥ metrics_only.** (14.40)
SRA trigger (no pooling): **SRA_NOW ⇔ [¬Meas_OK] ∨ [max ε_AB>0.05] ∨ [ρ<2] ∨ [E_align>0.2] ∨ [p̂<0.05].** (14.41)

Idempotent append: **Append(T,e) is a no-op if event_id(e) exists with identical hash.** (14.42)
Daily roll-ups: **R_day := MerkleRoot({h_t for day}); R_dataset := MerkleRoot({R_day}).** (14.43)

AB-fixedness (operational rule-of-thumb): **Commute_OK ∧ ρ≥2 ⇒ order-invariant majority on held-out (CSA@3 high).** (14.44)

Δ needle colors (defaults): **green if $\bar{\Delta} \le -0.2$; amber if $-0.2 < \bar{\Delta} < 0.2$; red if $\bar{\Delta} \ge 0.2$.** (14.45)

ESI two-light rule: *Commit ⇔ (CWA_OK = green) ∧ (χ ≤ χ), with χ≈0.6.*** (14.46)

A3. Thresholds & defaults (copy block)

*CSA@3 ≥ 0.67; max ε_AB ≤ 0.05; $\hat{p} \ge 0.05$; ρ ≥ 2.0; E_align ≤ 0.2; χ = 0.6; $\bar{\Delta}$bands = (−0.2, +0.2).** (14.47)

Starch budget S: 1% (low volatility), 2% (medium), 3% (high). (14.48)
Sous-vide passes: T_1 cool (0.3–0.5), T_2 warm (0.6–0.8), T_3 cool (0.2–0.4). (14.49)

PID safe start: K_p small; K_i = 0 until CSA stable; K_d for overshoot; rate-limit + saturate; gate by CWA_OK. (14.50)
LQR guard: large **R** for invasive actuators (TMS/tFUS); always gate by CWA_OK. (14.51)

Pre-reg endpoints (common): CSA@3, max ε_AB, $\hat{p}$, ρ, $\bar{\Delta}$(with $\hat{g}$,$\hat{\beta}$,$\hat{\gamma}$), ECE, κ/α, Λ_T, VerifyTrace, dataset_root_hash. (14.52)

A4. Report footers (uniform fields)

`CSA@3=… max ε=… p̂=… ρ=… ĝ/β̂/γ̂=… Δ̄=… S_max/h=… FitAUC=… ΔU=… ECE=… κ/α=… Λ_T=… g_neuro=… CI=… CL_sync=… env_hash=… seeds=… dataset_root_hash=…` (14.53)

A5. One-line reminders (to brief the team)

“You can’t unhappen your own write” → use **(14.2), (14.4), (14.5), (14.8)**. (14.54)
“Don’t average fighting thermometers” → require **(14.10)–(14.13)**. (14.55)
“One dial, one lamp” → watch **$\bar{\Delta}$** and **CWA_OK** (14.14, 14.13). (14.56)
“If χ clumps or ε spikes, cool/structure then retry” → **(14.29), (14.37), (14.46), (14.49).** (14.57)

End Appendix A.

14. Appendices — B. 10-Line Recipes (CSA/ε/CWA, Δ Estimators, Controllers)

> Paste as-is. Each block is exactly 10 lines, Blogger-ready Unicode with (14.Bx.n) tags. Thresholds match the CWA certificate.

B1. CSA/ε/CWA in 10 lines (order-swap agreement, interference, legality to average)

1. Build three **pure, non-mutating critics** O_1, O_2, O_3 with disjoint inputs (units • NLI-vs-

Given • trace-referencer). (14.B1.1)

2. For each item d, compute 2-of-3 majorities under all 6 orders; **CSA@3 := (1/N)·Σ 1{majority unchanged}.** (14.B1.2)
3. On a held-out set D, for each pair (A,B) estimate **ε_AB := Pr_{d∼D}[A∘B(d) ≠ B∘A(d)].** (14.B1.3)
4. Compute a scalar per case (score s_i); define **T_obs := | mean(s) − mean_π(s) |** where π shuffles grader order & phase. (14.B1.4)
5. **$\hat{p}$:= (1/B)·Σ_{b=1}^B 1{ | mean(s) − mean_{π_b}(s) | ≥ T_obs }.** (14.B1.5)
6. Track redundancy **ρ := fragments/claim** (text + time-locked sensor); require **ρ ≥ 2**. (14.B1.6)
7. **CWA_OK ⇔ [CSA@3 ≥ 0.67] ∧ [max ε_AB ≤ 0.05] ∧ [$\hat{p}$ ≥ 0.05].** (14.B1.7)
8. **If ¬CWA_OK ⇒ SRA only** (per-case reporting; no pooling/actuation). (14.B1.8)
9. Append one ledger event: **h_t := H(h_{t−1} ∥ canonical_json(e_t))** (tamper-evident). (14.B1.9)
10. Report footer: **CSA@3, max ε, $\hat{p}$, ρ, CWA_OK**, seeds, env_hash, dataset_root_hash. (14.B1.10)

B2. Δ estimators in 10 lines (push × echo − buffer, with CUSUM)

1. Prepare per-segment signals: **r_t** (guidance/“push”), **s_t** (stance shift), **jumps_t/ minutes_t**, **τ_recover**. (14.B2.1)
2. **$\hat{g}$:= cov(r,s) ÷ var(r)** (guidance slope; robust to scale). (14.B2.2)
3. **$\hat{\beta}$:= (1/T)·Σ_t (jumps_t ÷ minutes_t)** (amplification rate). (14.B2.3)
4. **$\hat{\gamma}$:= 1 ÷ τ_recover** (damping/buffer). (14.B2.4)
5. **Δ_t := $\hat{g}$_t·$\hat{\beta}$_t − $\hat{\gamma}$_t; $\bar{\Delta}$:= EMA_W(Δ_t).** (14.B2.5)
6. CUSUM: **S_t := max(0, S_{t−1} + Δ_t − τ)**; alarm if **S_t ≥ h**. (14.B2.6)
7. Bands: **green if $\bar{\Delta}$ ≤ −0.2; amber if −0.2<$\bar{\Delta}$<0.2; red if $\bar{\Delta}$ ≥ +0.2.** (14.B2.7)
8. Defaults: **α=0.05** (CWA), window **W=3**, log 95% CIs for $\hat{g}$,$\hat{\beta}$,$\hat{\gamma}$. (14.B2.8)
9. **Act only if CWA_OK**; else cool guidance or add redundancy, then recompute. (14.B2.9)
10. Append Δ event to Trace (hash-chain); include **$\bar{\Delta}$, S_max, τ,h** in footer. (14.B2.10)

B3. Controller stubs (CWA-gated)

B3-A. PID in 10 lines (model-light, safe by default)

1. **If ¬CWA_OK ⇒ u_t := u_hold; return.** (14.B3A.1)
2. Target dial: **Δ*** (e.g., −0.2); error **e_t := Δ* − Δ_t.** (14.B3A.2)
3. Integrator: **z_t := clamp(z_{t−1} + e_t·Δt, z_min, z_max)** (anti-windup). (14.B3A.3)
4. Derivative: **d_t := (e_t − e_{t−1}) ÷ Δt.** (14.B3A.4)
5. Raw control: **u_raw := K_p e_t + K_i z_t + K_d d_t.** (14.B3A.5)
6. Safety: **u_lim := rate_limit(u_raw; ρ_max)**; **u_t := sat_U(u_lim)** (bounds, IPIs). (14.B3A.6)
7. Defaults: **K_p small; K_i=0** until CSA stable; **K_d** to tame overshoot. (14.B3A.7)
8. Log **Δ̄, u_t**, guard states; if **CSA dips** or **ε spikes**, arm **u_hold**. (14.B3A.8)
9. Append actuation event to Trace (hash-chain). (14.B3A.9)
10. Publish only with **VerifyTrace=1** and **CWA_OK=green** in footer. (14.B3A.10)

B3-B. LQR in 10 lines (when a local model is available)

1. **If ¬CWA_OK ⇒ u_t := u_hold; return.** (14.B3B.1)
2. Identify local dynamics: **x_{t+1} ≈ A x_t + B u_t** (ridge on recent window). (14.B3B.2)
3. Choose costs: **Q ⪰ 0** (weight Δ/Δ-coords), **R ≻ 0** (actuator penalty). (14.B3B.3)
4. Solve Riccati (discrete): **K := dlqr(A,B,Q,R).** (14.B3B.4)
5. Policy: **u_raw := −K x_t.** (14.B3B.5)
6. Safety: **u_t := sat_U(rate_limit(u_raw)).** (14.B3B.6)
7. Re-ID A,B on a rolling window; freeze K if **CSA** dips or **ε** warms. (14.B3B.7)
8. Prefer large **R** for invasive actuators (TMS/tFUS). (14.B3B.8)
9. Hash-append controller step to Trace; store **Q,R,K,A,B** in tool card. (14.B3B.9)
10. Publish only with **CWA_OK=green** and **VerifyTrace=1**. (14.B3B.10)

Why these work. The CSA/ε/CWA block is the operational mirror of the observer results (latching + commuting + redundancy), Δ compresses push/echo/buffer into one actionable dial, and the controller stubs are thin, **CWA-gated** policies that respect safety and provenance (append-only, hash-chained Trace).

14. Appendices — C. Repro Pack (Seeds, Configs, Metrics Harness)

> Paste-as-is. Blogger-ready Unicode; single-line equations with (14.Cn) tags. Matches the SOPs and gates from §§8–13.

C1. Seeds & versions (what every run must record)

Seed vector (four knobs).
$\vec{s}$:= (s_global, s_split, s_solver, s_boot). (14.C1)

Environment tuple (hashable).
Env := (OS, CUDA/Driver, BLAS, Python, libs, commit). (14.C2)

Reproduce flag (metrics-tolerant).
Reproduce(D, $\vec{s}$, Env)=1 ⇔ ∀m∈M, |m′−m| ≤ tol_m, M={CSA@3, max ε, $\hat{p}$, $\hat{g}$, $\hat{\beta}$, $\hat{\gamma}$, $\bar{\Delta}$, ECE, κ/α, Λ_T}. (14.C3)

meta.json (drop-in).

```
{
  "dataset_version": "v1.0.0",
  "dataset_root_hash": "af21…",
  "env_hash": "sha256:…",
  "global_seed": 314159,
  "split_seed": 271828,
  "solver_seed": 161803,
  "boot_seed": 141421
}
```

(Seeds & versions logged alongside the dataset root hash.)

C2. Config file (paths, thresholds, seeds, switches)

configs/paper.yaml (copy-paste).

```
paths:
  data:        "/data"
  trace:       "/trace/trace.jsonl"
  graders_glob:"/trace/graders_O*.jsonl"
  out_dir:     "/dashboards"
thresholds:
  csa_min: 0.67
  eps_max: 0.05
  alpha_cwa: 0.05
  delta_bands: [-0.2, 0.2]
repro:
  env_hash: "sha256:…"
  seeds: {global:314159, split:271828, solver:161803, boot:141421}
report:
  include: [CSA, epsilon, CWA, rho, ghat, bhat, ghat_buf, Dbar, ECE, kappa, alpha, BF]
  footer_hashes: true
```

(Thresholds = **CWA certificate** defaults; footer prints hashes/seeds.)

C3. Metrics harness (one-screen CLI)

CSA (order-invariant majority).
`obs csa --graders /trace/graders_O*.jsonl --out /dashboards/csa.json` (14.C4)

ε (order-sensitivity heatmap).
`obs epsilon --graders /trace/graders_O*.jsonl --heldout /data/heldout.jsonl --out /dashboards/eps.json` (14.C5)

CWA permutation test (order + phase).
`obs cwa --scores /data/session_scores.json --B 1000 --out /dashboards/cwa.json` (14.C6)

Δ-dials (ĝ, β̂, γ̂, Δ̄, CUSUM).
`obs delta --segments /data/segments.jsonl --out /dashboards/delta.json` (14.C7)

Calibration & agreement extras.
`obs calib --probs probs.json --labels y.json --out ece.json` • `obs agree --labels graders.json --out kap_alpha.json` • `obs bf --liks H1.json H0.json --out bf.json` (14.C8)

Green-light rule (auto).
CWA_OK ⇔ [CSA@3 ≥ 0.67] ∧ [max ε_AB ≤ 0.05] ∧ [p̂ ≥ 0.05]. (14.C9)

C4. Deterministic data generation (synthetic fixtures)

Poisson jumps (amplification proxy).
assoc_jumps ~ Poisson(λ = topic_density × escalation_speed). (14.C10)

Affect AR(1) with scripted shocks (buffer proxy).
affect_t = μ + ϕ·affect_{t−1} + ε_t; recovery slope → γ̂ := 1 ÷ τ_recover. (14.C11)

Split recipe (stable splits).
train/dev/test = 70/15/15 with stratify by scenario × Δ-regime using split_seed. (14.C12)

C5. Δ estimators (the exact one-liners the harness uses)

Guidance slope: ĝ := cov(r,s) ÷ var(r). (14.C13)
Amplification rate: β̂ := (Σ_t jumps_t) ÷ (Σ_t minutes_t). (14.C14)
Buffer: γ̂ := 1 ÷ median_i(τ_recover,i). (14.C15)
Dial & EMA: Δ_t := ĝ_t·β̂_t − γ̂_t; Δ̄ := EMA_W(Δ_t). (14.C16)
CUSUM: S_t := max(0, S_{t−1} + Δ_t − τ); alarm if S_t ≥ h. (14.C17)

C6. Hashes & exports (tamper-evident by default)

Trace chain: h₀ := 0; h_t := H(h_{t−1} ∥ canonical_json(e_t)). (14.C18)
Merkle roll-ups: R_day := MerkleRoot({h_t}); R_dataset := MerkleRoot({R_day}). (14.C19)
Verify: VerifyTrace(T)=1 ⇔ recompute(h_T)=stored(h_T). (14.C20)
Export footer fields: `env_hash, seeds, dataset_root_hash, CSA@3, max ε, p̂, ĝ/β̂/γ̂, Δ̄, ECE, κ/α, Λ_T`. (14.C21)

C7. Unit tests that must pass (repro sanity)

1. **Idempotent append:** re-append same event → **h_T** unchanged. (14.C22)
2. **CSA invariance:** shuffling grader order leaves **CSA@3** stable on a fixed batch. (14.C23)
3. **ε sanity:** construct an interaction (O2 reads O1 output) → **ε_AB > 0**. (14.C24)

4. **Permutation fairness:** IID scores → $\hat{p} \approx 0.5$; order-coupled scores → $\hat{p} \to 0$. (14.C25)

5. **Seed reality:** rerun **obs delta** with same $\vec{s}$ → identical $\bar{\Delta}$ (within tol). (14.C26)

C8. One-command run (quickstart)

`obs repro --config /configs/paper.yaml --export /dashboards/report.pdf` (14.C27)
(Computes **CSA/ε/CWA**, **Δ-dials**, calibration, agreement, BF; emits a one-page report with hashes & seeds.)

C9. What "pass" looks like (acceptance bands)

CSA@3 ≥ 0.67; max ε_AB ≤ 0.05; $\hat{p} \geq 0.05$; $\bar{\Delta}$ in green (≤ −0.2); VerifyTrace=1; footer shows env_hash, seeds, dataset_root_hash. (14.C28)

C10. Notes & pointers (where each piece comes from)

- **Seeds/splits & synthetic fixtures:** Appendix D (seeds, Poisson/AR(1), split recipe).
- **CSA/ε/CWA gates:** Section 8 (definitions, permutation $\hat{p}$).
- **Δ & CUSUM:** Session-to-dashboard SOP (E.2–E.6).
- **Hashes/exports:** Governance appendix (F.3–F.6) + Quickstart.

Bottom line. Lock $\vec{s}$ and **Env**, run the four CLI calls, check the **CWA** lamp, and export with hashes. Anyone with your **meta.json** and config can rerun your figures and land on the same numbers. That's the whole point of this repro pack.

References

Core observer/measurement theory and ops

- *Self-Referential Observers in Quantum Dynamics: A Formal Theory of Internal Collapse and Cross-Observer Agreement*. [SRO] — Formal filtration/latching (delta-certainty), commutation, and AB-fixedness; the math backbone. https://osf.io/7cbsu/files/osfstorage/68c5961e10e31c4095d998f5

- *ObserverOps Technical Blueprint*. [OBS] — Engineering playbook: trace ledger, CSA/ε/CWA gates, APIs, dashboards. https://osf.io/yj5aw/files/osfstorage/68d30242dd3f77699b3c315f

- *Semantic Meme Field Theory (SMFT): Foundations, Projection, and Dynamics (Rev1)*. [SMFT] — Field layer linking continuous evolution with punctate collapse; frame invariance. https://osf.io/ya8tx/files/osfstorage/68e77fa0cd19895405a0d243

- *Emulsion-Stabilized Inference (ESI): Phase-Controlled Decoding with Structural "Starch" and Observer-Aligned Verification*. [ESI] — "Smoothness" (χ), starch budgets, sous-vide schedules; plugs into CSA/CWA.

https://osf.io/q8egv/files/osfstorage/68d58d6a5d44329625432c73

- *Semantic Collapse Geometry: A Unified Topological Model Linking Gödelian Logic, Attractor Dynamics, and Prime Number Gaps*. [SCG] — Collapse topology; invariants and attractor structure.
  https://osf.io/7jzpq

- *Proto-Eight Collapse Geometry—SMFT Applied to Growth, Memory, and Systems Built on Incubation Trigram (先天八卦)*. [P8] — Application of collapse geometry to growth/memory patterns.
  https://osf.io/ya8tx/files/osfstorage/68b84641534f31b42fef989e

Control, action, and variational foundations

- *AGI by Surplus-Aware Control: A Closed-Loop Framework of Surplus Flows, Semantic Field Geometry, and Dissipative Decoding*. [SAC] — Closed-loop surplus flows; links to $\Delta = \hat{g}\cdot\hat{\beta} - \hat{\gamma}$.
  https://osf.io/2wmky/files/osfstorage/68bd728b0fd5cbd1040356a2

- *From Entropy-Minimizing Attractor Proofs to Dissipative Lagrangian Dynamics: A Rigorous Foundation for the HeTu–LuoShu Variational Framework*. [HT-DL] — Variational/dissipative groundwork for the HeTu–LuoShu formalism.
  https://osf.io/2wmky/files/osfstorage/68b4d262a233f0f2da96aecd

- *A Generalized Least Action Principle for Local and Dissipative Systems: Axioms, Proof, and Domain of Validity*. [GLA] — Generalized action for non-conservative systems; domain where Δ-style control is valid.
  https://osf.io/2wmky/files/osfstorage/68b32a5ff4b17ecb9dc62067

- *The Slot Interpretation of HeTu and LuoShu: A Rigorous Mathematical and Semantic Proof by Wolfram 4.1 GPTs*
  https://osf.io/692wg/files/osfstorage/68960924847e9ead456b0e6c
  Δ5 Phase Opposition in HeTu: Pairwise Minimum-Dissipation Cycles and a D_{10}–Spectral Extension of the Slot Interpretation
  https://osf.io/38pw7/files/osfstorage/68e578b1dbe76397706d350d
  [HT-Slot] — Slot structure, phase opposition, spectral extensions for collapse scheduling.

Methods, coherence, and macro structure

- *CAFT + CWA + SRA: A Universal Additive Model of Macro Coherence (App A–F)*. [CAFT/CWA/SRA] — Coherence accounting; the "certificate to average" recipe and per-case reporting rules.
  https://osf.io/7cbsu/files/osfstorage/68a3065155d1ad4d6c7e40d4

- *Industrializing Insight: A Reproducible Method to Empower （灌顶加持）LLMs via the E=G+M+D Decomposition*. [E=G+M+D] — Practical instrumentation of guidance ($\hat{g}$), amplification ($\hat{\beta}$), damping ($\hat{\gamma}$).
  https://osf.io/6mybg/files/osfstorage/68d7dce87b362f1ca4b8f825

- *v1_01-05 System Topology Knowledge Base (txt)*. [STKB] — Glossary/topology snippets used across sections.
  https://osf.io/q8egv/files

- Belt Holonomy Is Inevitable_ A Two-Boundary Worldsheet from Standard Quantum Geometry
  https://osf.io/yaz5u/files/osfstorage/68cd926d1fd155818374e8f1
  Belt Theory for AGI — A Mini-Textbook
  https://osf.io/yaz5u/files/osfstorage/68cdecc75ecb719a6a413f46

Purpose-Flux Belt Theory (PFBT)
https://osf.io/yaz5u/files/osfstorage/68d01dd47195bb99223b7dfe
ONE Assumption 帶出宇宙「意圖」的必然性
https://gxstructure.blogspot.com/2025/09/one-assumption.html
— Macro-intent and purpose-flux notes (contextual background).

Consciousness/awareness geometry

- The Geometry of Awareness: Designing Semantic Collapse in AI Systems
https://www.amazon.com/dp/B0F8NSFKKM
意識原本: 重構語義、模因與AI自我之源代碼 (Traditional Chinese Edition)
https://www.amazon.com/dp/B0F8D32ZJD
[GoA] — Emergence of Ô (observer) as a projection solution in nonlinear dynamics.

- Trace之道：語義 AI 的設計哲學與未來教育場景重構
https://gxstructure.blogspot.com/2025/04/trace-ai.html
Ô Mirror：數位阿賴耶識與語義投胎系統設計白皮書
https://gxstructure.blogspot.com/2025/04/o-mirror.html
[Ālaya] — Applications of deep memory store constructs in observer terms.

- Unified Field Theory of Everything - Ch1~22 Appendix A~D
https://osf.io/ya8tx/files/osfstorage/68ed687e6ca51f0161dc3c55
[UFTE] — Broad unification backdrop; used here only for notation/intuition.

- *Nested Uplifts Inevitability: A Sequential-Evidence and Small-Gain Theory of Regime Switching in Open Dissipative Systems*
https://osf.io/ne89a/files/osfstorage/68effd340c8fad784bc40616
Nested Uplifts Inevitability (INU) Assumption 3.3 and the Riemann Hypothesis: Engineering Relaxations, Conceptual Bridges, and What Current Evidence Allows
https://osf.io/y98bc/files/osfstorage/68f0afbacaed018c3cc3fd9b
The Birth of Arithmetic Stability Theory: A Mathematical Bridge Rooted in Confucian Symmetry and Balance
https://fieldtheoryofeverything.blogspot.com/2025/10/the-birth-of-arithmetic-stability.html
從「黎曼猜想」到 - 觀察的本質就是「譜塌縮」
https://gxstructure.blogspot.com/2025/10/blog-post_17.html
[INU/RS] — Regime switching and spectral-collapse notes (used for stress-test design).

Bridging to brain science & humanities (the three on-topic companions)

- *The Observer And The Observed: A Unified Theory of Consciousness Across Biological Systems*. [O&O] — PFC as meta-observer; claustrum as binder; proposed empirical batteries.
https://thegrandredesign.substack.com/api/v1/file/fd4ad912-ba1f-439b-a7e2-e472580c8851.pdf

- *From Psychoanalytic Constructs to Closed-Loop Control: A Rigorous Mathematical Recast of Freud via Observer-Centric Collapse*. [Freud→Control] — Operators Ŝ_tight, U_⊥, R_θ, V^→; safe scheduling in Δ-space.
https://osf.io/w6be2/files/osfstorage/68f3d5d48a8dd1325519ff88

- *Wittgenstein, Operationalized: A Unified Mathematical Framework for Picture Theory, Language Games, and Hinge Certainty*. [Witt-Ops] — CSP picture-fit, IRL "meaning = use," BF hinge switching.
https://osf.io/tjf59/files/osfstorage/68f2c1745bd9c41be2f98369

© 2025 Danny Yeung. All rights reserved. 版权所有 不得转载

Disclaimer

This book is the product of a collaboration between the author and OpenAI's GPT-5, Wolfram's GPTs language model. While every effort has been made to ensure accuracy, clarity, and insight, the content is generated with the assistance of artificial intelligence and may contain factual, interpretive, or mathematical errors. Readers are encouraged to approach the ideas with critical thinking and to consult primary scientific literature where appropriate.

This work is speculative, interdisciplinary, and exploratory in nature. It bridges metaphysics, physics, and organizational theory to propose a novel conceptual framework—not a definitive scientific theory. As such, it invites dialogue, challenge, and refinement.

I am merely a midwife of knowledge.